Comprehensive Code Documentation

CSU2220 - Generative AI for Building Applications

AI Implementation Analysis

December 19, 2025

Contents

1 Original Code: code1.py

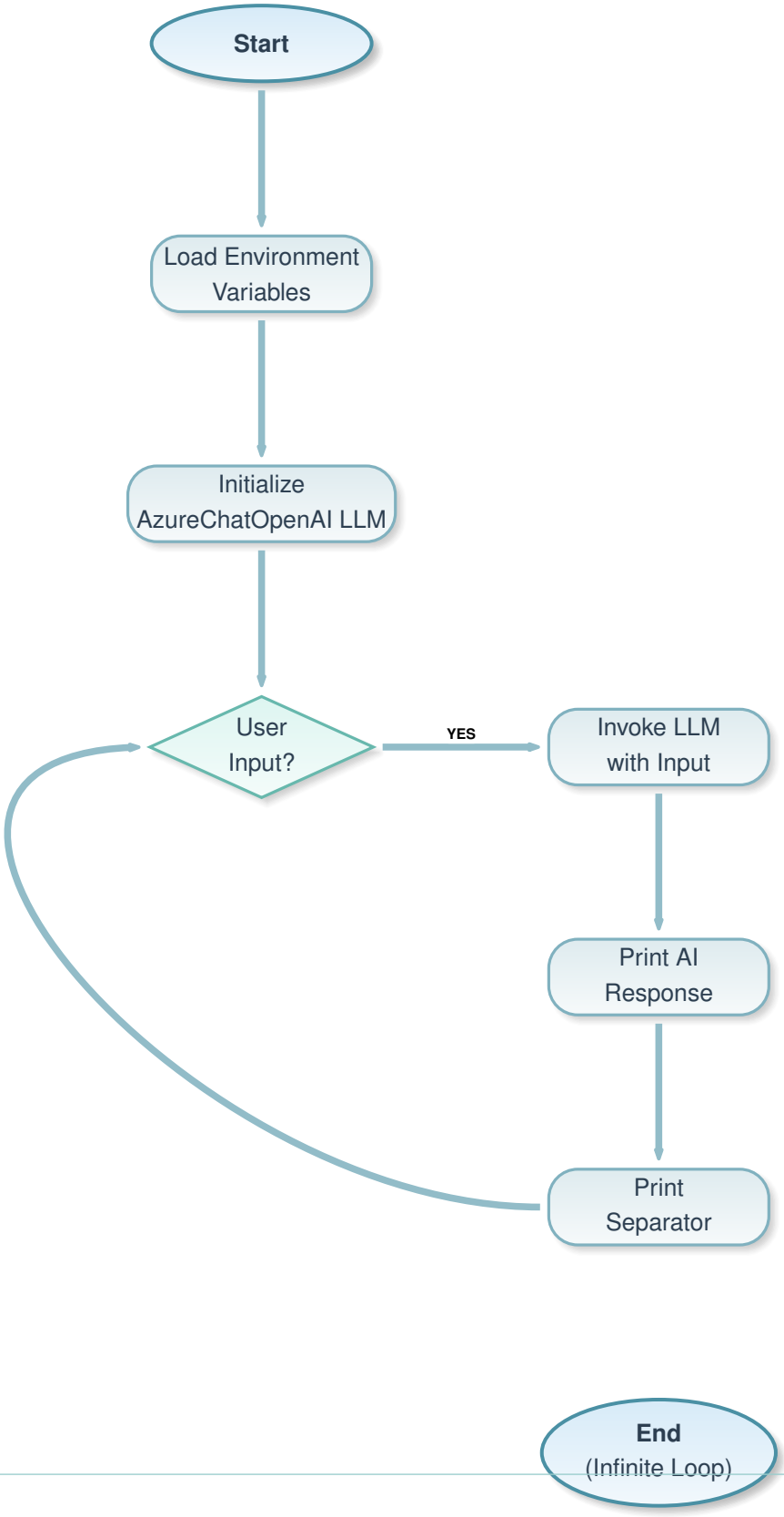
```python
from langchain.chat_models import AzureChatOpenAI
import os
from dotenv import load_dotenv


load_dotenv()

llm = AzureChatOpenAI(
    openai_api_base=os.getenv("AZURE_OPENAI_API_BASE"),
    openai_api_version=os.getenv("AZURE_OPENAI_API_VERSION"),
    openai_api_key=os.getenv("AZURE_OPENAI_API_KEY"),
    deployment_name=os.getenv("AZURE_OPENAI_DEPLOYMENT_NAME"),
    model_name="gpt-4o",
    temperature=0.7,
)

while True:
    input_text=input("Enter your query: ")
    result = llm.invoke(input_text)
    print("AI:",result.content)
    print("-------------------------------------------------")
```

1.1 Explanation of the Code

- **What:** This code implements a simple interactive chatbot that continuously takes user queries as input and generates responses using an Azure OpenAI model via LangChain. It prints the AI's response and separates interactions with a line.
- **Why:** The code demonstrates basic integration with cloud-based AI models like Azure OpenAI for conversational AI. It uses LangChain for abstraction, dotenv for secure environment variable management, and a loop for continuous interaction. This approach is chosen for simplicity in prototyping AI chatbots, ensuring API keys are not hardcoded, and leveraging LangChain's unified interface for different LLMs.
- **How:**
  - Load environment variables from a .env file using dotenv to securely access API credentials.

- Initialize an AzureChatOpenAI instance with the loaded credentials, specifying the model (gpt-4o) and temperature (0.7 for creativity).
- Enter an infinite loop: Prompt the user for input, invoke the LLM with the input to generate a response, print the AI's content from the result object, and print a separator line.

1.2 Flowchart



2 Ollama Equivalent Code

```python
from langchain_ollama import OllamaLLM

llm = OllamaLLM(model="qwen3:4b", temperature=0.7)

while True:
    input_text = input("Enter your query: ")
    result = llm.invoke(input_text)
    print("AI:", result)
    print("--------------------------------------------------")
```

2.1 Diff Explanation

— Removed Code

```
- from langchain.chat_models import AzureChatOpenAI
- import os
- from dotenv import load_dotenv
- load_dotenv()
- llm = AzureChatOpenAI(
-     openai_api_base=os.getenv("AZURE_OPENAI_API_BASE"),
-     openai_api_version=os.getenv("AZURE_OPENAI_API_VERSION"),
-     openai_api_key=os.getenv("AZURE_OPENAI_API_KEY"),
-     openai_api_version=os.getenv("AZURE_OPENAI_DEPLOYMENT_NAME"),
-     model_name="gpt-4o",
-     temperature=0.7,
- )
```

+++ Added Code

```
+ from langchain_ollama import OllamaLLM
+ llm = OllamaLLM(model="qwen3:4b", temperature=0.7)
```

Modified Code

print("AI:", result.content) -> print("AI:", result)

2.2 Explanation of the Diff

- **Removed imports and initialization:** The Azure-specific imports and environment variable loading are removed because Ollama runs locally and doesn't require API keys or cloud configuration.
- **Changed LLM initialization:** Switched to OllamaLLM for local model execution, using a specific model name "qwen3:4b" instead of Azure's gpt-4o.
- **Modified print statement:** In Ollama, the result is directly the string, not an object with .content, so no need to access .content.
- **Why:** These changes adapt the code from cloud-based Azure OpenAI to local Ollama deployment, simplifying setup by removing authentication and using local model inference.

3 Hugging Face Equivalent Code

```python
from transformers import pipeline

pipe = pipeline("text-generation", model="Qwen/Qwen3-0.6B")

while True:
    input_text = input("Enter your query: ")
    result = pipe(input_text, max_length=100, temperature=0.7,
    ↪ num_return_sequences=1)
    print("AI:", result[0]['generated_text'])
    print("-------------------------------------------------")
```

3.1 Diff Explanation

— Removed Code

```
- from langchain.chat_models import AzureChatOpenAI
- import os
- from dotenv import load_dotenv
- load_dotenv()
- llm = AzureChatOpenAI(...)
- result = llm.invoke(input_text)
- print("AI:",result.content)
```

+++ Added Code

```
+ from transformers import pipeline
+ pipe = pipeline("text-generation", model="Qwen/Qwen3-0.6B")
+ result = pipe(input_text, max_length=100, temperature=0.7,
+              num_return_sequences=1)
+ print("AI:", result[0]['generated_text'])
```

3.2 Explanation of the Diff

- **Removed LangChain and Azure setup:** Replaced with Hugging Face transformers for direct model interaction.
- **Changed to pipeline:** Uses text-generation pipeline with a specific model, adding parameters like max_length and num_return_sequences for generation control.

- **Modified result handling:** The result is a list of dictionaries, accessing the generated text accordingly.
- **Why:** Hugging Face provides a different API focused on transformers models, requiring different initialization and invocation methods, and handles generation parameters explicitly.

4 Original Code: code2.py

```python
import os
from dotenv import load_dotenv
from langchain_openai import AzureChatOpenAI

# Load environment variables
load_dotenv()

# Fetch from .env or directly assign as needed
api_key = ""
api_base = "https://chatgpt-key.openai.azure.com/"
deployment_name = "gpt-4o-2"
api_version = "2024-12-01-preview"  # Ensure this is a valid API version

print(f"API Key: {api_key}")

# Initialize the model
llm = AzureChatOpenAI(
    deployment_name=deployment_name,
    api_version=api_version,
    api_key=api_key,
    azure_endpoint=api_base,
    model_name="gpt-4o",
    temperature=0.7,
    max_tokens=100,  # Maximum number of tokens in the response
)


# Use the predict method instead of chat
response = llm.predict("Hello, how are you?")

# Print the response
print(response)
```

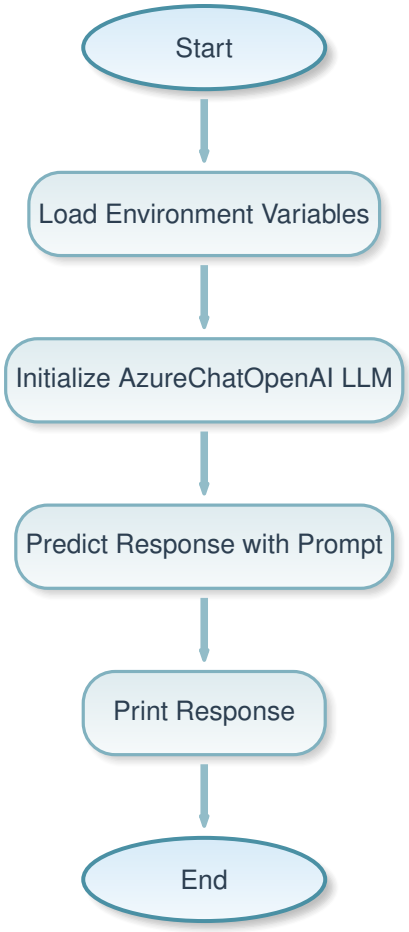
4.1 Explanation of the Code

- **What:** This code initializes an Azure OpenAI model using LangChain and generates a single response to the prompt "Hello, how are you?" using the predict method, then prints the response.
- **Why:** It demonstrates basic setup and single-query interaction with Azure OpenAI via

LangChain. The use of predict method shows an alternative to invoke for simple text generation. Environment variables are loaded for security, though here API key is empty (placeholder). This approach is for testing or simple API calls without continuous interaction.

- **How:**
  - Load environment variables (though API key is hardcoded empty here).
  - Print the API key (for debugging).
  - Initialize AzureChatOpenAI with deployment details, API version, key, endpoint, model name, temperature, and max tokens.
  - Call predict with a fixed prompt to get a response.
  - Print the response string.

4.2 Flowchart



5 Ollama Equivalent Code

```python
from langchain_ollama import OllamaLLM

llm = OllamaLLM(
    model="qwen3:4b",
    temperature=0.7,
    max_tokens=100,
)

response = llm.invoke("Hello, how are you?")

print(response)
```

5.1 Diff Explanation

— Removed Code

```
- import os
- from dotenv import load_dotenv
- from langchain_openai import AzureChatOpenAI
- load_dotenv()
- api_key = ""
- api_base = "https://chatgpt-key.openai.azure.com/"
- deployment_name = "gpt-4o-2"
- api_version = "2024-12-01-preview"
- print(f"API Key: {api_key}")
- llm = AzureChatOpenAI(...)
- response = llm.predict("Hello, how are you?")
```

+++ Added Code

```
+ from langchain_ollama import OllamaLLM
+ llm = OllamaLLM(model="qwen3:4b", temperature=0.7, max_tokens=100)
```

Modified Code

response = llm.invoke("Hello, how are you?")

5.2 Explanation of the Diff

- **Removed Azure setup:** Eliminated environment loading, API key handling, and Azure-specific parameters since Ollama is local.
- **Changed initialization:** Used OllamaLLM with model name and parameters.
- **Modified invocation:** Switched from predict to invoke, as Ollama uses invoke for generation.
- **Why:** Adapts to Ollama's local execution, removing cloud dependencies and using LangChain's Ollama wrapper for consistency.

6 Hugging Face Equivalent Code

```python
from transformers import pipeline

pipe = pipeline("text-generation", model="Qwen/Qwen3-0.6B")

response = pipe("Hello, how are you?", max_new_tokens=50, temperature=0.7,
    ↪  num_return_sequences=1)

print(response[0]['generated_text'])
```

6.1 Diff Explanation

— Removed Code

```
- import os
- from dotenv import load_dotenv
- from langchain_openai import AzureChatOpenAI
- load_dotenv()
- api_key = ""
- ... (Azure init)
- response = llm.predict("Hello, how are you?")
- print(response)
```

+++ Added Code

```
+ from transformers import pipeline
+ pipe = pipeline("text-generation", model="Qwen/Qwen3-0.6B")
+ response = pipe("Hello, how are you?", max_new_tokens=50, temperature=0.7,
num_return_sequences=1)
+ print(response[0]['generated_text'])
```

6.2 Explanation of the Diff

- **Removed LangChain and Azure:** Replaced with direct Hugging Face pipeline.
- **Changed to pipeline generation:** Used text-generation pipeline with max_new_tokens instead of max_tokens.
- **Modified output handling:** Accessed the generated text from the result list.
- **Why:** Hugging Face transformers provide a different interface for model loading and generation, focusing on local model execution without LangChain abstraction.

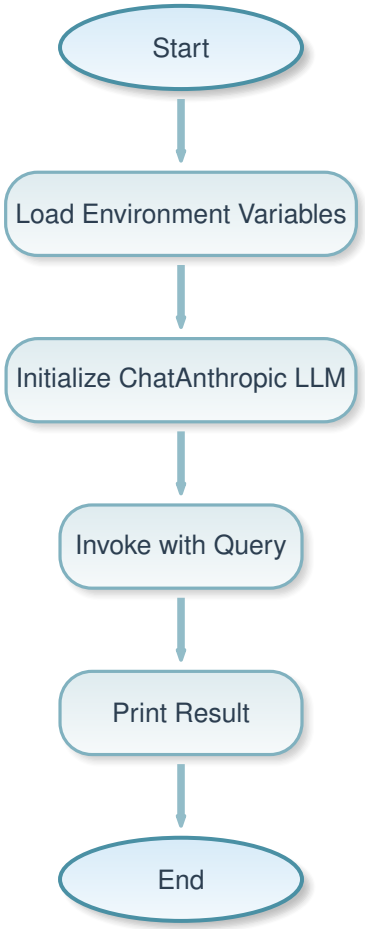
7 Original Code: code3.py

```python
#ChatAnthropic - cluade

from langchain_anthropic import ChatAnthropic
from dotenv import load_dotenv
import os
load_dotenv()
llm=ChatAnthropic(temperature=0.7, model="claude-2", anthropic_api_key=os.
    ↪ getenv("ANTHROPIC_API_KEY"))
result=llm.invoke("What is the capital of France?")
print(result)
```

7.1 Explanation of the Code

- **What:** This code initializes a ChatAnthropic model (Claude) using LangChain and generates a response to the query "What is the capital of France?", then prints the result.
- **Why:** Demonstrates integration with Anthropic's Claude model via LangChain for AI-powered question answering. Uses environment variables for API key security. This approach is for simple, single-turn interactions with proprietary models like Claude.
- **How:**
  - Load environment variables from .env file.
  - Initialize ChatAnthropic with temperature, model name, and API key from env.
  - Invoke the model with a fixed question.
  - Print the result object (which contains the response).

7.2 Flowchart



8 Ollama Equivalent Code

```python
from langchain_ollama import OllamaLLM

llm = OllamaLLM(model="qwen3:4b", temperature=0.7)

result = llm.invoke("What is the capital of France?")

print(result)
```

8.1 Diff Explanation

— Removed Code

```
- from langchain_anthropic import ChatAnthropic
- from dotenv import load_dotenv
- import os
- load_dotenv()
- llm=ChatAnthropic(temperature=0.7, model="claude-2",
anthropic_api_key=os.getenv("ANTHROPIC_API_KEY"))
```

+++ Added Code

```
+ from langchain_ollama import OllamaLLM
+ llm = OllamaLLM(model="qwen3:4b", temperature=0.7)
```

8.2 Explanation of the Diff

- **Removed Anthropic setup:** Eliminated API key loading and ChatAnthropic initialization.
- **Changed to Ollama:** Used OllamaLLM for local execution.
- **Why:** Switches from cloud-based Anthropic API to local Ollama model, removing authentication requirements.

9 Hugging Face Equivalent Code

```python
from transformers import pipeline

pipe = pipeline("text-generation", model="Qwen/Qwen3-0.6B")

result = pipe("What is the capital of France?", max_new_tokens=50,
    ↪ temperature=0.7, num_return_sequences=1)

print(result[0]['generated_text'])
```

9.1 Diff Explanation

— Removed Code

```
- from langchain_anthropic import ChatAnthropic
- from dotenv import load_dotenv
- import os
- load_dotenv()
- llm=ChatAnthropic(...)
- result=llm.invoke("What is the capital of France?")
- print(result)
```

+++ Added Code

```
+ from transformers import pipeline
+ pipe = pipeline("text-generation", model="Qwen/Qwen3-0.6B")
+ result = pipe("What is the capital of France?", max_new_tokens=50, temperature=0.7,
num_return_sequences=1)
+ print(result[0]['generated_text'])
```

9.2 Explanation of the Diff

- **Removed LangChain Anthropic:** Replaced with Hugging Face pipeline.
- **Changed invocation and output:** Used pipeline generate method and accessed text from result.
- **Why:** Adapts to Hugging Face's direct model interface for text generation.

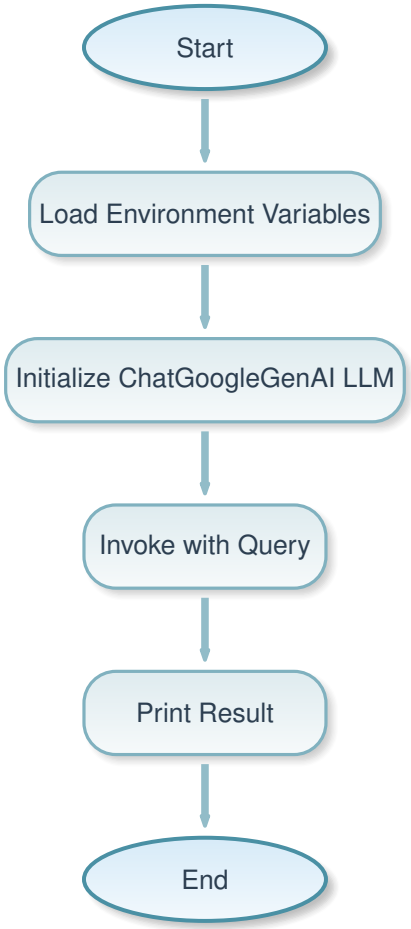
10 Original Code: code4.py

```python
from langchain_google_genai import ChatGoogleGenAI
from dotenv import load_dotenv
import os
load_dotenv()

llm=ChatGoogleGenAI(temperature=0.7, model="gemini-1.5-pro",
    ↪ google_api_key=os.getenv("GOOGLE_API_KEY"))


result=llm.invoke("What is the capital of France?")
print(result)
```

10.1 Explanation of the Code

- **What:** This code initializes a ChatGoogleGenAI model (Gemini) using LangChain and generates a response to "What is the capital of France?", then prints it.
- **Why:** Shows integration with Google's Gemini model via LangChain for AI responses. Uses env vars for API key. Suitable for single queries to Google's AI.
- **How:**
  - Load env vars.
  - Init ChatGoogleGenAI with temp, model, API key.
  - Invoke with query.
  - Print result.

10.2 Flowchart



11 Ollama Equivalent Code

```python
from langchain_ollama import OllamaLLM

llm = OllamaLLM(model="qwen3:4b", temperature=0.7)

result = llm.invoke("What is the capital of France?")

print(result)
```

11.1 Diff Explanation

— Removed Code

```
- from langchain_google_genai import ChatGoogleGenAI
- from dotenv import load_dotenv
- import os
- load_dotenv()
- llm=ChatGoogleGenAI(temperature=0.7, model="gemini-1.5-pro",
google_api_key=os.getenv("GOOGLE_API_KEY"))
```

+++ Added Code

```
+ from langchain_ollama import OllamaLLM
+ llm = OllamaLLM(model="qwen3:4b", temperature=0.7)
```

11.2 Explanation of the Diff

- **Removed Google setup:** Eliminated API key and ChatGoogleGenAI.
- **Changed to Ollama:** Local model execution.
- **Why:** From cloud Google API to local Ollama.

12 Hugging Face Equivalent Code

```python
from transformers import pipeline

pipe = pipeline("text-generation", model="Qwen/Qwen3-0.6B")

result = pipe("What is the capital of France?", max_new_tokens=50,
    ↪ temperature=0.7, num_return_sequences=1)

print(result[0]['generated_text'])
```

12.1 Diff Explanation

— Removed Code

```
- from langchain_google_genai import ChatGoogleGenAI
- ...
- result=llm.invoke("What is the capital of France?")
- print(result)
```

+++ Added Code

```
+ from transformers import pipeline
+ pipe = pipeline("text-generation", model="Qwen/Qwen3-0.6B")
+ result = pipe("What is the capital of France?", max_new_tokens=50, temperature=0.7,
num_return_sequences=1)
+ print(result[0]['generated_text'])
```

12.2 Explanation of the Diff

- **Removed LangChain Google:** To Hugging Face pipeline.
- **Changed invocation:** Pipeline generate.
- **Why:** Direct transformers usage.

13 Original Code: code5.py

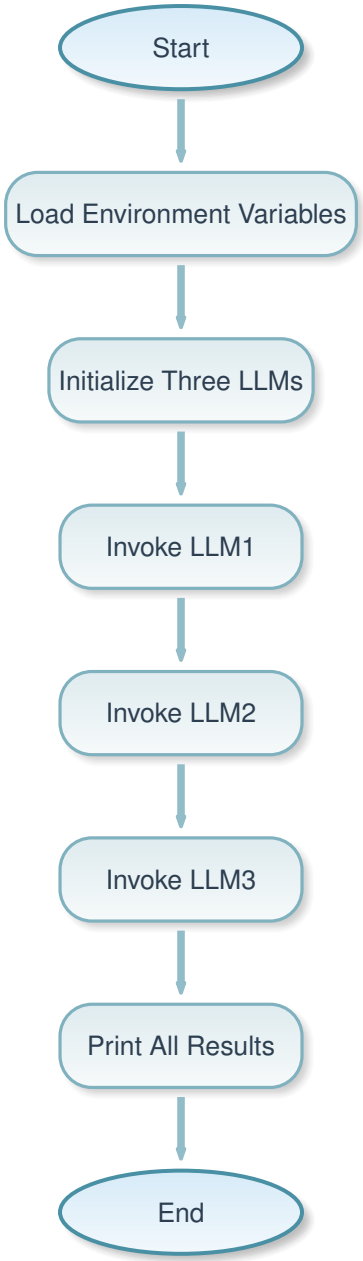
```python
from langchain_openai import ChatOpenAI

from langchain_google_genai import ChatGoogleGenAI

from langchain_anthropic import ChatAnthropic

from dotenv import load_dotenv
import os


llm1=ChatOpenAI(model="gpt-3.5-turbo",temperature=0.5,openai_api_key=os.
    ↪ getenv("OPENAI_API_KEY"))
llm2=ChatGoogleGenAI(temperature=0.7, model="gemini-1.5-pro",
    ↪ google_api_key=os.getenv("GOOGLE_API_KEY"))
llm3=ChatAnthropic(temperature=0.7, model="claude-2", anthropic_api_key=os
    ↪ .getenv("ANTHROPIC_API_KEY"))

result1=llm1.invoke("Opinion on Pakistan?")
result2=llm2.invoke("Opinion on Pakistan?")
result3=llm3.invoke("Opinion on Pakistan?")

print("OpenAI result: ",result1)
print("Google result: ",result2)
print("Anthropic result: ",result3)
```

13.1 Explanation of the Code

- **What:** This code initializes three different LLM models (OpenAI GPT-3.5, Google Gemini, Anthropic Claude) using LangChain and generates responses to the same query "Opinion on Pakistan?" from each, then prints them labeled by provider.
- **Why:** Demonstrates comparing outputs from multiple AI providers for the same prompt, highlighting differences in responses. Uses env vars for API keys. This is useful for evaluating model performance or biases across platforms.
- **How:**
  - Load env vars.
  - Initialize three LLMs with different providers, models, and temps.

- Invoke each with the same query.
- Print results with labels.

13.2 Flowchart



14 Ollama Equivalent Code

```python
from langchain_ollama import OllamaLLM

llm1 = OllamaLLM(model="qwen3:4b", temperature=0.5)
llm2 = OllamaLLM(model="qwen3:4b", temperature=0.7)  # Change model name to ones installed for better experience
llm3 = OllamaLLM(model="qwen3:4b", temperature=0.7)  # Change model name to ones installed for better experience

result1 = llm1.invoke("Opinion on Pakistan?")
result2 = llm2.invoke("Opinion on Pakistan?")
result3 = llm3.invoke("Opinion on Pakistan?")

print("OpenAI result: ", result1)
print("Google result: ", result2)
print("Anthropic result: ", result3)
```

14.1 Diff Explanation

— Removed Code

```
- from langchain_openai import ChatOpenAI
- from langchain_google_genai import ChatGoogleGenAI
- from langchain_anthropic import ChatAnthropic
- from dotenv import load_dotenv
- import os
- llm1=ChatOpenAI(model="gpt-3.5-turbo",temperature=0.5,openai_api_key=os.getenv("OPENAI_API_KEY"))
- llm2=ChatGoogleGenAI(temperature=0.7, model="gemini-1.5-pro", google_api_key=os.getenv("GOOGLE_API_KEY"))
- llm3=ChatAnthropic(temperature=0.7, model="claude-2", anthropic_api_key=os.getenv("ANTHROPIC_API_KEY"))
```

+++ Added Code

```
+ from langchain_ollama import OllamaLLM
+ llm1 = OllamaLLM(model="qwen3:4b", temperature=0.5)
+ llm2 = OllamaLLM(model="qwen3:4b", temperature=0.7)
+ llm3 = OllamaLLM(model="qwen3:4b", temperature=0.7)
```

14.2 Explanation of the Diff

- **Removed multiple provider setups:** Eliminated API keys and different LangChain imports.
- **Changed to multiple Ollama instances:** Used same model with varying temperatures for comparison.
- **Why:** Adapts to local Ollama, simulating multi-model comparison with different configs instead of different providers.

15 Hugging Face Equivalent Code

```python
from transformers import pipeline

pipe1 = pipeline("text-generation", model="Qwen/Qwen3-0.6B")
pipe2 = pipeline("text-generation", model="Qwen/Qwen3-0.6B")  # Change model name to ones installed for better experience
pipe3 = pipeline("text-generation", model="Qwen/Qwen3-0.6B")  # Change model name to ones installed for better experience

result1 = pipe1("Opinion on Pakistan?", max_new_tokens=50, temperature=0.5, num_return_sequences=1)
result2 = pipe2("Opinion on Pakistan?", max_new_tokens=50, temperature=0.7, num_return_sequences=1)
result3 = pipe3("Opinion on Pakistan?", max_new_tokens=50, temperature=0.7, num_return_sequences=1)

print("OpenAI result: ", result1[0]['generated_text'])
print("Google result: ", result2[0]['generated_text'])
print("Anthropic result: ", result3[0]['generated_text'])
```

15.1 Diff Explanation

— Removed Code

```
- from langchain_openai import ChatOpenAI
- ...
- llm1=...
- llm2=...
- llm3=...
```

```
- result1=llm1.invoke("Opinion on Pakistan?")
- result2=llm2.invoke("Opinion on Pakistan?")
- result3=llm3.invoke("Opinion on Pakistan?")
- print("OpenAI result: ",result1)
- print("Google result: ",result2)
- print("Anthropic result: ",result3)
```

+++ Added Code

```
+ from transformers import pipeline
+ pipe1 = pipeline("text-generation", model="Qwen/Qwen3-0.6B")
+ pipe2 = pipeline("text-generation", model="Qwen/Qwen3-0.6B")
+ pipe3 = pipeline("text-generation", model="Qwen/Qwen3-0.6B")
+ result1 = pipe1("Opinion on Pakistan?", max_new_tokens=50, temperature=0.5,
num_return_sequences=1)
+ result2 = pipe2("Opinion on Pakistan?", max_new_tokens=50, temperature=0.7,
num_return_sequences=1)
+ result3 = pipe3("Opinion on Pakistan?", max_new_tokens=50, temperature=0.7,
num_return_sequences=1)
+ print("OpenAI result: ", result1[0]['generated_text'])
+ print("Google result: ", result2[0]['generated_text'])
+ print("Anthropic result: ", result3[0]['generated_text'])
```

15.2 Explanation of the Diff

- **Removed LangChain multi-provider:** Replaced with multiple Hugging Face pipelines.
- **Changed invocation and output:** Used pipeline generate and accessed text.
- **Why:** Uses direct transformers for local multi-instance comparison.

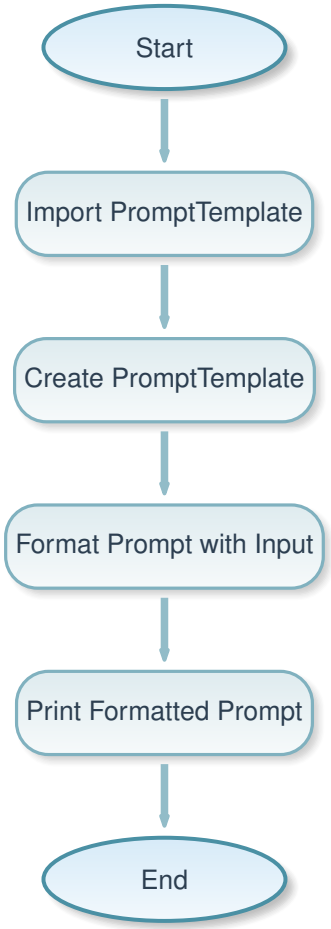
16 Original Code: code6.py

```python
from langchain.prompts import PromptTemplate

prompt=PromptTemplate(
    input_variables=["input"],
    template="What is the capital of {input}?",
)

print(prompt.format(input="China"))
```

16.1 Explanation of the Code

- **What:** This code defines a PromptTemplate with a placeholder for input, formats it with "China", and prints the resulting prompt string.
- **Why:** Demonstrates LangChain's PromptTemplate for dynamic prompt creation. Useful for reusable prompts with variables. Here, it just formats and prints without invoking an LLM.
- **How:**
  - Import PromptTemplate.
  - Create template with input_variables and template string.
  - Format with specific input.
  - Print the formatted string.

16.2 Flowchart



17 Ollama Equivalent Code

```python
from langchain_core.prompts import PromptTemplate
from langchain_ollama import OllamaLLM

# Initialize the LLM with Ollama
llm = OllamaLLM(model="qwen3:4b")

prompt = PromptTemplate(
    input_variables=["input"],
    template="What is the capital of {input}?",
)

# Create a chain
```

```
chain = prompt | llm

# Invoke the chain
result = chain.invoke({"input": "China"})
print(result)
```

17.1 Diff Explanation

— Removed Code

```
- from langchain.prompts import PromptTemplate
- prompt=PromptTemplate(...)
- print(prompt.format(input="China"))
```

+++ Added Code

```
+ from langchain_core.prompts import PromptTemplate
+ from langchain_ollama import OllamaLLM
+ llm = OllamaLLM(model="qwen3:4b")
+ chain = prompt | llm
+ result = chain.invoke({"input": "China"})
+ print(result)
```

17.2 Explanation of the Diff

- **Added LLM and chaining:** Introduced OllamaLLM and chain composition with | operator.
- **Changed invocation:** From formatting only to invoking the chain with dict input.
- **Why:** Extends to actual LLM invocation using LangChain's chaining for prompt + model.

18 Hugging Face Equivalent Code

```
from transformers import pipeline

# Initialize the pipeline with Qwen model
pipe = pipeline("text-generation", model="Qwen/Qwen3-0.6B")

# Format the prompt
prompt = "What is the capital of China?"

```

```
# Generate text
result = pipe(prompt, max_new_tokens=50)
print(result[0]['generated_text'])
```

18.1 Diff Explanation

— Removed Code

```
- from langchain.prompts import PromptTemplate
- prompt=PromptTemplate(...)
- print(prompt.format(input="China"))
```

+++ Added Code

```
+ from transformers import pipeline
+ pipe = pipeline("text-generation", model="Qwen/Qwen3-0.6B")
+ prompt = "What is the capital of China?"
+ result = pipe(prompt, max_new_tokens=50)
+ print(result[0]['generated_text'])
```

18.2 Explanation of the Diff

- **Removed PromptTemplate:** Manual string formatting.
- **Added pipeline generation:** Direct text generation with pipeline.
- **Why:** Hugging Face doesn't use LangChain templates, so manual prompt creation and pipeline invocation.

19 Original Code: code7.py

```python
from langchain.prompts import PromptTemplate

from langchain.chat_models import AzureChatOpenAI

from langchain.chains import LLMChain
from dotenv import load_dotenv
import os
load_dotenv()
# Fetch from .env
api_key = os.getenv("AZURE_OPENAI_API_KEY")
api_base = os.getenv("AZURE_OPENAI_ENDPOINT")
deployment_name = os.getenv("AZURE_OPENAI_DEPLOYMENT_NAME")
api_version = os.getenv("AZURE_OPENAI_API_VERSION")
llm=AzureChatOpenAI(
    deployment_name=deployment_name,
    api_version=api_version,
    api_key=api_key,
    azure_endpoint=api_base,
    model_name="gpt-4",
    temperature=0.9,
    top_p=0.9,
    max_tokens=100,
# Maximum number of tokens in the response
)
promt=PromptTemplate(
    input_variables=["country"],
    template="what your opinion on this country {country}?",
)
chain=LLMChain(llm=llm,prompt=promt)
result=chain.invoke({"country":"India"})
print(result['text'])
```

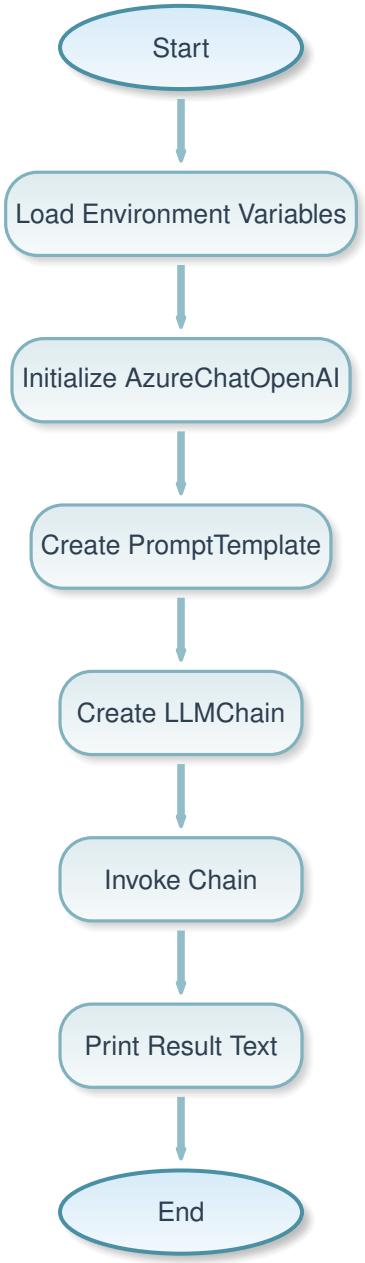
19.1 Explanation of the Code

- **What:** This code creates a prompt template, initializes an Azure OpenAI LLM, combines them into an LLMChain, invokes the chain with "India" as input, and prints the generated text.
- **Why:** Demonstrates LangChain's chaining mechanism to combine prompts and LLMs for dynamic query generation. Uses Azure for cloud AI, with parameters like temperature and

top_p for response control.

- **How:**
  - Load env vars for Azure credentials.
  - Init AzureChatOpenAI with params.
  - Create PromptTemplate.
  - Create LLMChain with llm and prompt.
  - Invoke chain with dict input.
  - Print result['text'].

19.2 Flowchart



20 Ollama Equivalent Code

```
from langchain_core.prompts import PromptTemplate
from langchain_ollama import OllamaLLM
```

```

# Initialize the LLM with Ollama
llm = OllamaLLM(model="qwen3:4b", temperature=0.9)

prompt = PromptTemplate(
    input_variables=["country"],
    template="what your opinion on this country {country}?",
)

# Create a chain
chain = prompt | llm

# Invoke the chain
result = chain.invoke({"country": "India"})
print(result)
```

20.1 Diff Explanation

— Removed Code

```
- from langchain.prompts import PromptTemplate
- from langchain.chat_models import AzureChatOpenAI
- from langchain.chains import LLMChain
- from dotenv import load_dotenv
- import os
- load_dotenv()
- api_key = os.getenv("AZURE_OPENAI_API_KEY")
- ... (Azure init)
- promt=PromptTemplate(...)
- chain=LLMChain(llm=llm,prompt=promt)
- result=chain.invoke({"country":"India"})
- print(result['text'])
```

+++ Added Code

```
+ from langchain_core.prompts import PromptTemplate
+ from langchain_ollama import OllamaLLM
+ llm = OllamaLLM(model="qwen3:4b", temperature=0.9)
+ prompt = PromptTemplate(...)
+ chain = prompt | llm
+ result = chain.invoke({"country": "India"})
+ print(result)
```

20.2 Explanation of the Diff

- **Removed Azure setup and old LLMChain:** Eliminated env loading and AzureChatOpenAI.
- **Changed to modern chaining:** Used | operator instead of LLMChain class.
- **Simplified result access:** Direct print of result instead of result['text'].
- **Why:** Updates to newer LangChain syntax and adapts to Ollama local model.

21 Hugging Face Equivalent Code

```python
from transformers import pipeline

# Initialize the pipeline with Qwen model
pipe = pipeline("text-generation", model="Qwen/Qwen3-0.6B")

# Format the prompt
prompt = "what your opinion on this country India?"

# Generate text
result = pipe(prompt, max_new_tokens=100, temperature=0.9, top_p=0.9)
print(result[0]['generated_text'])
```

21.1 Diff Explanation

— Removed Code

```
- from langchain.prompts import PromptTemplate
- ... (Azure and chain setup)
- result=chain.invoke({"country":"India"})
- print(result['text'])
```

+++ Added Code

```
+ from transformers import pipeline
+ pipe = pipeline("text-generation", model="Qwen/Qwen3-0.6B")
+ prompt = "what your opinion on this country India?"
+ result = pipe(prompt, max_new_tokens=100, temperature=0.9, top_p=0.9)
+ print(result[0]['generated_text'])
```

21.2 Explanation of the Diff

- **Removed LangChain components:** No templates or chains.
- **Manual prompt and pipeline:** Direct string prompt and generation.
- **Why:** Hugging Face focuses on direct model interaction without LangChain abstractions.

22 Original Code: code8.py

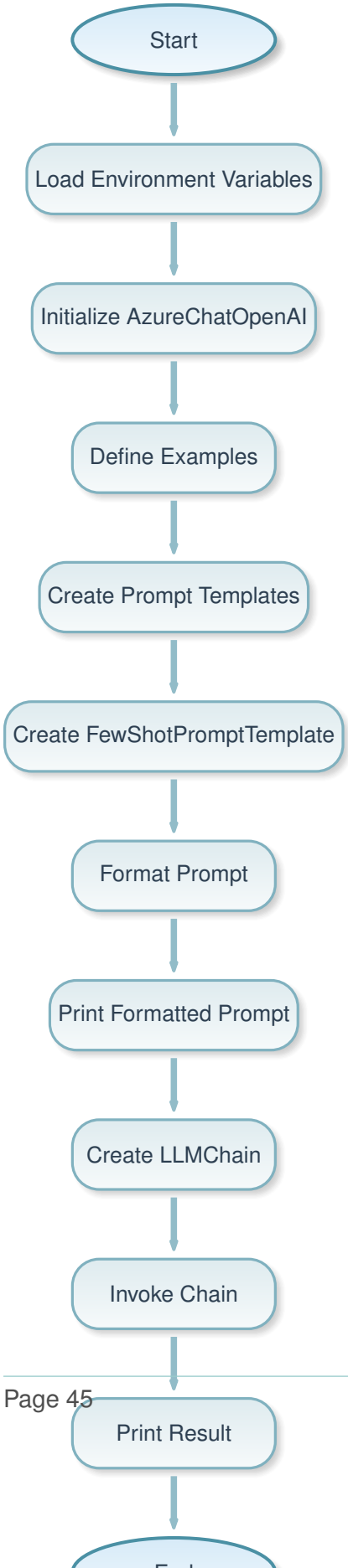
```python
from langchain.chat_models import AzureChatOpenAI
from langchain.prompts import FewShotPromptTemplate, PromptTemplate
from dotenv import load_dotenv
from langchain.chains import LLMChain
import os
load_dotenv()

api_key = os.getenv("AZURE_OPENAI_API_KEY")
api_base = os.getenv("AZURE_OPENAI_ENDPOINT")
deployment_name = os.getenv("AZURE_OPENAI_DEPLOYMENT_NAME")
api_version = os.getenv("AZURE_OPENAI_API_VERSION")

llm=AzureChatOpenAI(
    deployment_name=deployment_name,
    api_version=api_version,
    api_key=api_key,
    azure_endpoint=api_base,
    model_name="gpt-4",
    temperature=0.9,
    top_p=0.9,
    max_tokens=100,
# Maximum number of tokens in the response
)
examples = [
{"question": "What is the capital of France?", "answer": "The capital of
    ↪ France is Paris."},
{"question": "What is the capital of Germany?", "answer": "The capital of
    ↪ Germany is Berlin."},
{"question": "What is the capital of Italy?", "answer": "The capital of
    ↪ Italy is Rome."},
]

examples_prompt = PromptTemplate(
    input_variables=["question", "answer"],
    template="Question: {question}\nAnswer: {answer}",

)

few_shot_prompt = FewShotPromptTemplate(
    examples=examples,
    examples_prompt=examples_prompt,
    input_variables=["question"],
```

```
    suffix="Answer:",
    prefix="You are a helpful assistant. Answer the following question.",
)

formatted_prompt = few_shot_prompt.format(question="What is the capital of
    ↪ Spain?")
print(formatted_prompt)

llmchain=LLMChain(
    llm=llm,
    prompt=few_shot_prompt,
)
result=llmchain.invoke({"question":"What is the capital of Spain?"})
print(formatted_prompt)
print(result['text'])
```

22.1 Explanation of the Code

- **What:** This code sets up a few-shot prompt template with example Q&A pairs, formats it for a new question, prints the formatted prompt, then uses an LLMChain with Azure OpenAI to generate and print the response.
- **Why:** Demonstrates few-shot learning by providing examples in the prompt to guide the model's response. Useful for tasks requiring specific formats or reasoning patterns. Uses Azure for scalable AI.
- **How:**
  - Load env vars.
  - Init AzureChatOpenAI.
  - Define examples list.
  - Create examples_prompt template.
  - Create FewShotPromptTemplate with examples, prefix, suffix.
  - Format the prompt with new question and print.
  - Create LLMChain and invoke with question.
  - Print formatted prompt again and result['text'].

22.2 Flowchart



23 Ollama Equivalent Code

```python
from langchain_core.prompts import FewShotPromptTemplate, PromptTemplate
from langchain_ollama import OllamaLLM

# Initialize the LLM with Ollama
llm = OllamaLLM(model="qwen3:4b", temperature=0.9)

examples = [
    {"question": "What is the capital of France?", "answer": "The capital of France is Paris."},
    {"question": "What is the capital of Germany?", "answer": "The capital of Germany is Berlin."},
    {"question": "What is the capital of Italy?", "answer": "The capital of Italy is Rome."},
]

examples_prompt = PromptTemplate(
    input_variables=["question", "answer"],
    template="Question: {question}\nAnswer: {answer}",
)

few_shot_prompt = FewShotPromptTemplate(
    examples=examples,
    example_prompt=examples_prompt,
    input_variables=["question"],
    suffix="Answer:",
    prefix="You are a helpful assistant. Answer the following question.",
)

# Create a chain
chain = few_shot_prompt | llm

# Invoke the chain
result = chain.invoke({"question": "What is the capital of Spain?"})
print(result)
```

23.1 Diff Explanation

— Removed Code

```
- from langchain.chat_models import AzureChatOpenAI
- from langchain.prompts import FewShotPromptTemplate, PromptTemplate
- from dotenv import load_dotenv
- from langchain.chains import LLMChain
- import os
- load_dotenv()
- api_key = ... (Azure setup)
- llm=AzureChatOpenAI(...)
- formatted_prompt = few_shot_prompt.format(question="What is the capital of Spain?")
- print(formatted_prompt)
- llmchain=LLMChain(llm=llm, prompt=few_shot_prompt)
- result=llmchain.invoke({"question":"What is the capital of Spain?"})
- print(formatted_prompt)
- print(result['text'])
```

+++ Added Code

```
+ from langchain_core.prompts import FewShotPromptTemplate, PromptTemplate
+ from langchain_ollama import OllamaLLM
+ llm = OllamaLLM(model="qwen3:4b", temperature=0.9)
+ chain = few_shot_prompt | llm
+ result = chain.invoke({"question": "What is the capital of Spain?"})
+ print(result)
```

23.2 Explanation of the Diff

- **Removed Azure and old imports:** Eliminated cloud setup.
- **Updated to modern LangChain:** Used langchain_core and | chaining.
- **Simplified output:** Removed duplicate prints and direct result printing.
- **Why:** Adapts few-shot prompting to Ollama with updated syntax.

24 Hugging Face Equivalent Code

```python
from transformers import pipeline

# Initialize the pipeline with Qwen model
```

```python
pipe = pipeline("text-generation", model="Qwen/Qwen3-0.6B")

# Build the few-shot prompt manually
examples = [
    {"question": "What is the capital of France?", "answer": "The capital of France is Paris."},
    {"question": "What is the capital of Germany?", "answer": "The capital of Germany is Berlin."},
    {"question": "What is the capital of Italy?", "answer": "The capital of Italy is Rome."},
]

prompt = "You are a helpful assistant. Answer the following question.\n"
for ex in examples:
    prompt += f"Question: {ex['question']}\nAnswer: {ex['answer']}\n"
prompt += "Question: What is the capital of Spain?\nAnswer:"

# Generate text
result = pipe(prompt, max_new_tokens=100, temperature=0.9, top_p=0.9)
print(result[0]['generated_text'])
```

24.1 Diff Explanation

— Removed Code

```
- from langchain.chat_models import AzureChatOpenAI
- ... (all LangChain setup)
- formatted_prompt = ...
- print(formatted_prompt)
- llmchain=...
- result=...
- print(...)
```

+++ Added Code

```
+ from transformers import pipeline
+ pipe = pipeline("text-generation", model="Qwen/Qwen3-0.6B")
+ prompt = "You are a helpful assistant. Answer the following question.\n"
+ for ex in examples:
+     prompt += f"Question: {ex['question']}
Answer: {ex['answer']}
"
+ prompt += "Question: What is the capital of Spain?\nAnswer:"
+ result = pipe(prompt, max_new_tokens=100, temperature=0.9, top_p=0.9)
```

```
+ print(result[0]['generated_text'])
```

24.2 Explanation of the Diff

- **Removed LangChain abstractions:** No templates or chains.
- **Manual prompt construction:** Built the few-shot prompt as a string.
- **Direct pipeline use:** Generated text directly.
- **Why:** Hugging Face requires manual prompt engineering for few-shot learning.

25 Original Code: code9.py

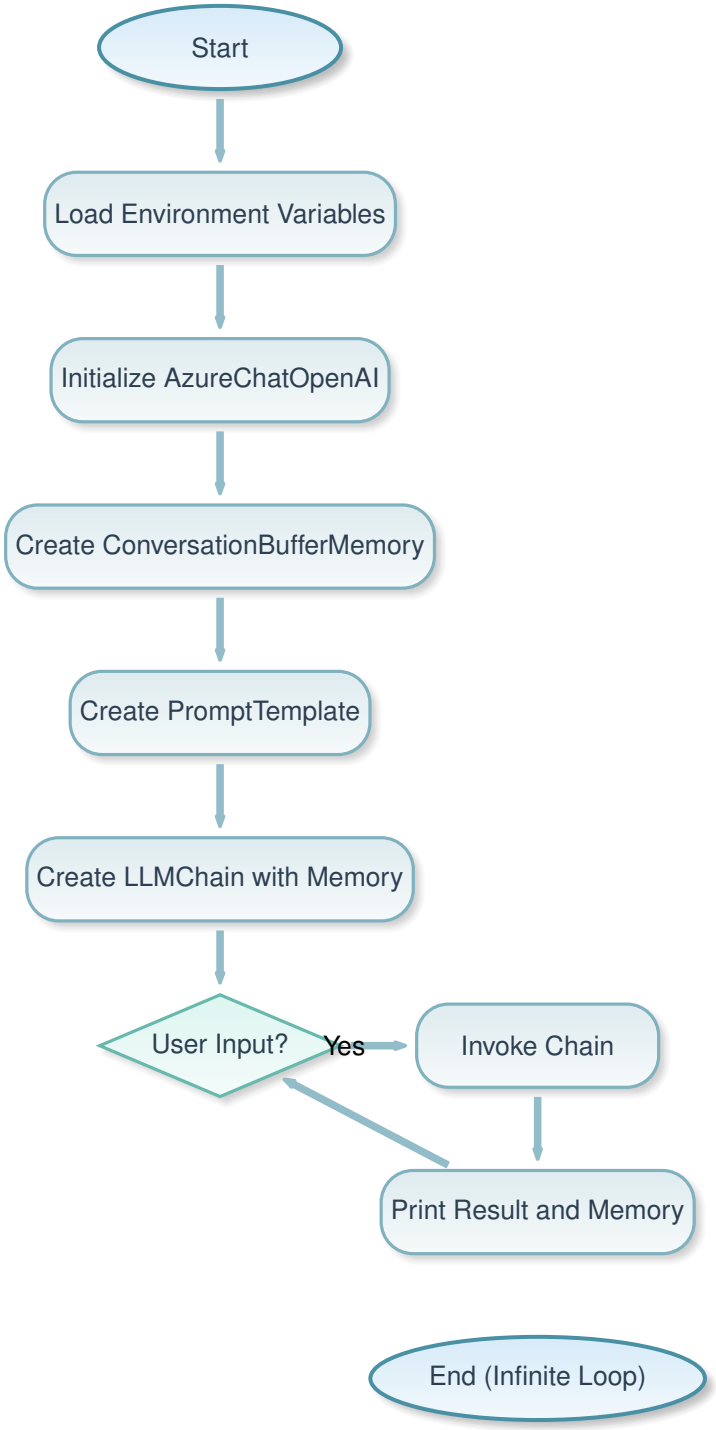
```python
from langchain.chat_models import AzureChatOpenAI
import os
from dotenv import load_dotenv
load_dotenv()
from langchain.chains import LLMChain
from langchain.prompts import PromptTemplate
from langchain.memory import ConversationBufferMemory

llm_object = AzureChatOpenAI(
    openai_api_base=os.getenv("AZURE_OPENAI_API_BASE"),
    openai_api_version=os.getenv("AZURE_OPENAI_API_VERSION"),
    openai_api_key=os.getenv("AZURE_OPENAI_API_KEY"),
    deployment_name=os.getenv("AZURE_OPENAI_DEPLOYMENT_NAME"),
    model_name="gpt-4o",
    temperature=0.7,
)

memory = ConversationBufferMemory(
    memory_key="chat_history",
    return_messages=True,
)

prompt = PromptTemplate(
    input_variables=["chat_history", "user_input"],
    template="""

   "summary in 50 words technical

    Conversation history:
    {chat_history}
    User: {user_input}
    AI:
    """
)

chain = LLMChain(
    llm=llm_object,
    prompt=prompt,
    memory=memory
)

while True:
```

```
43      inp = input("Enter the query: ")
44      result = chain.invoke({"user_input": inp})
45      print(result['text'])
46      print(memory.buffer)
```

25.1 Explanation of the Code

- **What:** This code sets up a conversational AI with memory using Azure OpenAI, LangChain's ConversationBufferMemory, and a prompt that includes chat history. It runs in a loop taking user input, generating responses with context, and printing the response and memory buffer.
- **Why:** Demonstrates persistent conversation memory in AI chatbots, allowing the model to recall previous interactions. Useful for coherent multi-turn dialogues. Uses Azure for reliable cloud AI.
- **How:**
  - Load env vars.
  - Init AzureChatOpenAI.
  - Create ConversationBufferMemory.
  - Define prompt with chat_history and user_input.
  - Create LLMChain with llm, prompt, memory.
  - Loop: Get input, invoke chain (memory auto-updates), print result['text'] and memory.buffer.

25.2 Flowchart



26 Ollama Equivalent Code

```python
from langchain_ollama import OllamaLLM
from langchain_core.prompts import PromptTemplate
from langchain_classic.memory import ConversationBufferMemory

llm = OllamaLLM(model="qwen3:4b", temperature=0.7)

memory = ConversationBufferMemory(
    memory_key="chat_history",
    return_messages=True,
)

prompt = PromptTemplate(
    input_variables=["chat_history", "user_input"],
    template="""

   "summary in 50 words technical

    Conversation history:
    {chat_history}
    User: {user_input}
    AI:
    """
)

# Using pipe operator instead of deprecated LLMChain
chain = prompt | llm

while True:
    inp = input("Enter the query: ")
    result = chain.invoke({"user_input": inp, "chat_history": memory.
    ↪ buffer})
    print(result)
    memory.save_context({"input": inp}, {"output": result})
```

26.1 Diff Explanation

— Removed Code

```
- from langchain.chat_models import AzureChatOpenAI
- import os
- from dotenv import load_dotenv
- load_dotenv()
- from langchain.chains import LLMChain
- from langchain.prompts import PromptTemplate
- from langchain.memory import ConversationBufferMemory
- llm_object = AzureChatOpenAI(...)
- chain = LLMChain(llm=llm_object, prompt=prompt, memory=memory)
- result = chain.invoke({"user_input": inp})
- print(result['text'])
- print(memory.buffer)
```

+++ Added Code

```
+ from langchain_ollama import OllamaLLM
+ from langchain_core.prompts import PromptTemplate
+ from langchain_classic.memory import ConversationBufferMemory
+ llm = OllamaLLM(model="qwen3:4b", temperature=0.7)
+ chain = prompt | llm
+ result = chain.invoke({"user_input": inp, "chat_history": memory.buffer})
+ print(result)
+ memory.save_context({"input": inp})
```

26.2 Explanation of the Diff

- **Removed Azure setup:** No env vars or cloud API.
- **Updated imports:** Used langchain_core and langchain_classic for memory.
- **Changed chaining:** | operator instead of LLMChain.
- **Manual memory update:** Since | chain doesn't auto-update memory, save_context is called manually.
- **Why:** Adapts conversational memory to Ollama with modern LangChain, requiring explicit memory management.

27 Hugging Face Equivalent Code

```
from transformers import pipeline

pipe = pipeline("text-generation", model="Qwen/Qwen3-0.6B")

chat_history = []

while True:
    inp = input("Enter the query: ")
    history_str = "\n".join(chat_history)
    prompt = f"""

   "summary in 50 words technical

    Conversation history:
    {history_str}
    User: {inp}
    AI:
    """
    result = pipe(prompt, max_new_tokens=100, temperature=0.7,
    ↪ num_return_sequences=1)
    response = result[0]['generated_text'].split("AI:")[-1].strip() if "AI
    ↪ :" in result[0]['generated_text'] else result[0]['generated_text']
    print(response)
    chat_history.append(f"User: {inp}")
    chat_history.append(f"AI: {response}")
    print(chat_history)
```

27.1 Diff Explanation

— Removed Code

```
- from langchain.chat_models import AzureChatOpenAI
- ... (all LangChain components)
- memory = ConversationBufferMemory(...)
- chain = LLMChain(...)
- result = chain.invoke({"user_input": inp})
- print(result['text'])
- print(memory.buffer)
```

+++ Added Code

```
+ from transformers import pipeline
+ pipe = pipeline("text-generation", model="Qwen/Qwen3-0.6B")
+ chat_history = []
+ history_str = "\n".join(chat_history)
+ prompt = f"..." (manual prompt building)
+ result = pipe(prompt, max_new_tokens=100, temperature=0.7, num_return_sequences=1)
+ response = result[0]['generated_text'].split("AI:")[-1].strip() ...
+ print(response)
+ chat_history.append(f"User: {inp}"
+ chat_history.append(f"AI: {response}"
+ print(chat_history)
```

27.2 Explanation of the Diff

- **Removed LangChain memory:** No built-in memory management.
- **Manual history tracking:** Used list to store and build prompt.
- **Prompt engineering:** Constructed prompt string manually with history.
- **Response parsing:** Extracted response from generated text.
- **Why:** Hugging Face requires custom implementation for conversational memory, as it lacks LangChain's abstractions.

28 Original Code: code9A.py

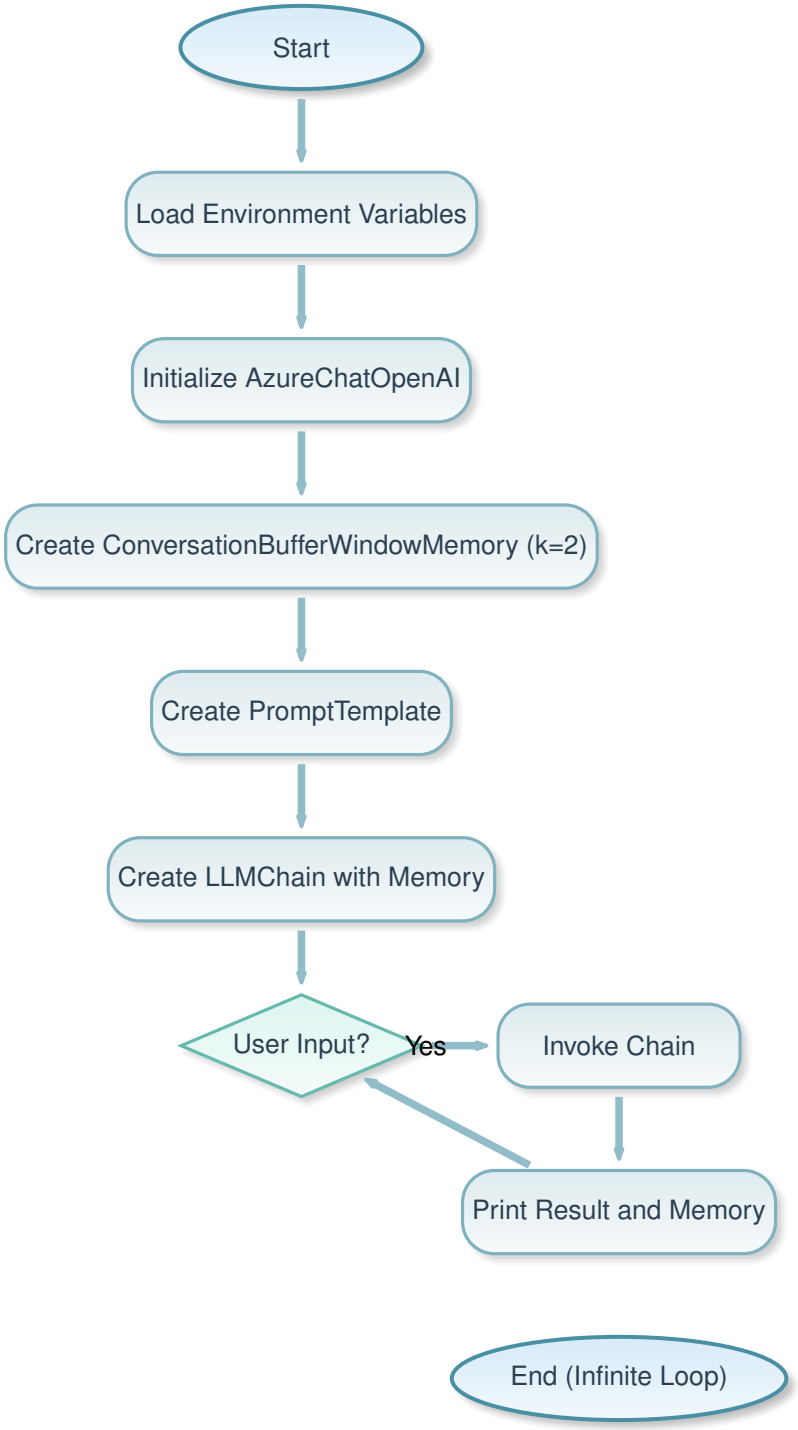
```python
from langchain.chat_models import AzureChatOpenAI
import os
from dotenv import load_dotenv
load_dotenv()
from langchain.chains import LLMChain
from langchain.prompts import PromptTemplate
from langchain.memory import ConversationBufferWindowMemory

llm_object = AzureChatOpenAI(
    openai_api_base=os.getenv("AZURE_OPENAI_API_BASE"),
    openai_api_version=os.getenv("AZURE_OPENAI_API_VERSION"),
    openai_api_key=os.getenv("AZURE_OPENAI_API_KEY"),
    deployment_name=os.getenv("AZURE_OPENAI_DEPLOYMENT_NAME"),
    model_name="gpt-4o",
    temperature=0.7,
)

memory = ConversationBufferWindowMemory(
    memory_key="chat_history",
    return_messages=True,
    k=2
)

prompt = PromptTemplate(
    input_variables=["chat_history", "user_input"],
    template="""

   "summary in 50 words technical

    Conversation history:
    {chat_history}
    User: {user_input}
    AI:
    """
)

chain = LLMChain(
    llm=llm_object,
    prompt=prompt,
    memory=memory
)

```

```
while True:
    inp = input("Enter the query: ")
    result = chain.invoke({"user_input": inp})
    print(result['text'])
    print(memory.buffer)
```

28.1 Explanation of the Code

- **What:** This code implements a conversational AI with windowed memory (keeping only the last 2 interactions) using Azure OpenAI and LangChain's ConversationBufferWindowMemory. It loops for user input, generates context-aware responses, and prints the result and memory buffer.
- **Why:** Demonstrates memory management to prevent context from growing indefinitely, focusing on recent history for efficiency. Useful for long conversations where old context becomes less relevant. Uses Azure for consistent AI performance.
- **How:**
  - Load env vars.
  - Init AzureChatOpenAI.
  - Create ConversationBufferWindowMemory with k=2 (last 2 turns).
  - Define prompt with history and input.
  - Create LLMChain with memory.
  - Loop: Input, invoke, print result['text'] and memory.buffer.

28.2 Flowchart



29 Ollama Equivalent Code

```python
from langchain_ollama import OllamaLLM
from langchain_core.prompts import PromptTemplate
from langchain_classic.memory import ConversationBufferWindowMemory

llm_object = OllamaLLM(model="qwen3:4b", temperature=0.7)

memory = ConversationBufferWindowMemory(
    memory_key="chat_history",
    return_messages=True,
    k=2
)

prompt = PromptTemplate(
    input_variables=["chat_history", "user_input"],
    template="""

   "summary in 50 words technical

    Conversation history:
    {chat_history}
    User: {user_input}
    AI:
    """
)

# Using pipe operator instead of deprecated LLMChain
chain = prompt | llm_object

while True:
    inp = input("Enter the query: ")
    result = chain.invoke({"user_input": inp, "chat_history": memory.
    ↪ buffer})
    print(result)
    memory.save_context({"input": inp}, {"output": result})
```

29.1 Diff Explanation

— Removed Code

```
- from langchain.chat_models import AzureChatOpenAI
- import os
- from dotenv import load_dotenv
- load_dotenv()
- from langchain.chains import LLMChain
- from langchain.prompts import PromptTemplate
- from langchain.memory import ConversationBufferWindowMemory
- llm_object = AzureChatOpenAI(...)
- chain = LLMChain(llm=llm_object, prompt=prompt, memory=memory)
- result = chain.invoke({"user_input": inp})
- print(result['text'])
- print(memory.buffer)
```

+++ Added Code

```
+ from langchain_ollama import OllamaLLM
+ from langchain_core.prompts import PromptTemplate
+ from langchain_classic.memory import ConversationBufferWindowMemory
+ llm_object = OllamaLLM(model="qwen3:4b", temperature=0.7)
+ chain = prompt | llm_object
+ result = chain.invoke({"user_input": inp, "chat_history": memory.buffer})
+ print(result)
+ memory.save_context({"input": inp})
```

29.2 Explanation of the Diff

- **Removed Azure setup:** Local Ollama instead.
- **Updated imports:** langchain_core and langchain_classic.
- **Modern chaining:** | operator.
- **Manual memory save:** Required for | chains.
- **Why:** Adapts windowed memory to Ollama with updated LangChain patterns.

30 Hugging Face Equivalent Code

```
1 from transformers import pipeline
```

```

pipe = pipeline("text-generation", model="Qwen/Qwen3-0.6B")

chat_history = []

while True:
    inp = input("Enter the query: ")
    # Keep only last 2 exchanges (4 messages: 2 user + 2 AI)
    if len(chat_history) > 4:
        chat_history = chat_history[-4:]
    history_str = "\n".join(chat_history)
    prompt = f"""

   "summary in 50 words technical

    Conversation history:
    {history_str}
    User: {inp}
    AI:
    """
    result = pipe(prompt, max_new_tokens=100, temperature=0.7,
    ↪ num_return_sequences=1)
    response = result[0]['generated_text'].split("AI:")[-1].strip() if "AI
    ↪ :" in result[0]['generated_text'] else result[0]['generated_text']
    print(response)
    chat_history.append(f"User: {inp}")
    chat_history.append(f"AI: {response}")
    print(chat_history)
```

30.1 Diff Explanation

— Removed Code

```
- from langchain.chat_models import AzureChatOpenAI
- ... (all LangChain)
- memory = ConversationBufferWindowMemory(...)
- chain = LLMChain(...)
- result = chain.invoke({"user_input": inp})
- print(result['text'])
- print(memory.buffer)
```

+++ Added Code

```
+ from transformers import pipeline
+ pipe = pipeline("text-generation", model="Qwen/Qwen3-0.6B")
+ chat_history = []
+ if len(chat_history) > 4: chat_history = chat_history[-4:]
+ history_str = "\n".join(chat_history)
+ prompt = f"..." (manual)
+ result = pipe(prompt, max_new_tokens=100, temperature=0.7, num_return_sequences=1)
+ response = result[0]['generated_text'].split("AI:")[-1].strip() ...
+ print(response)
+ chat_history.append(f"User: {inp}"
+ chat_history.append(f"AI: {response}"
+ print(chat_history)
```

30.2 Explanation of the Diff

- **Removed LangChain memory:** Custom window implementation.
- **Manual windowing:** Truncate history to last 4 messages.
- **Prompt building:** String concatenation for history.
- **Why:** Hugging Face requires manual memory and prompt management for windowed conversations.

31 Original Code: code9B.py

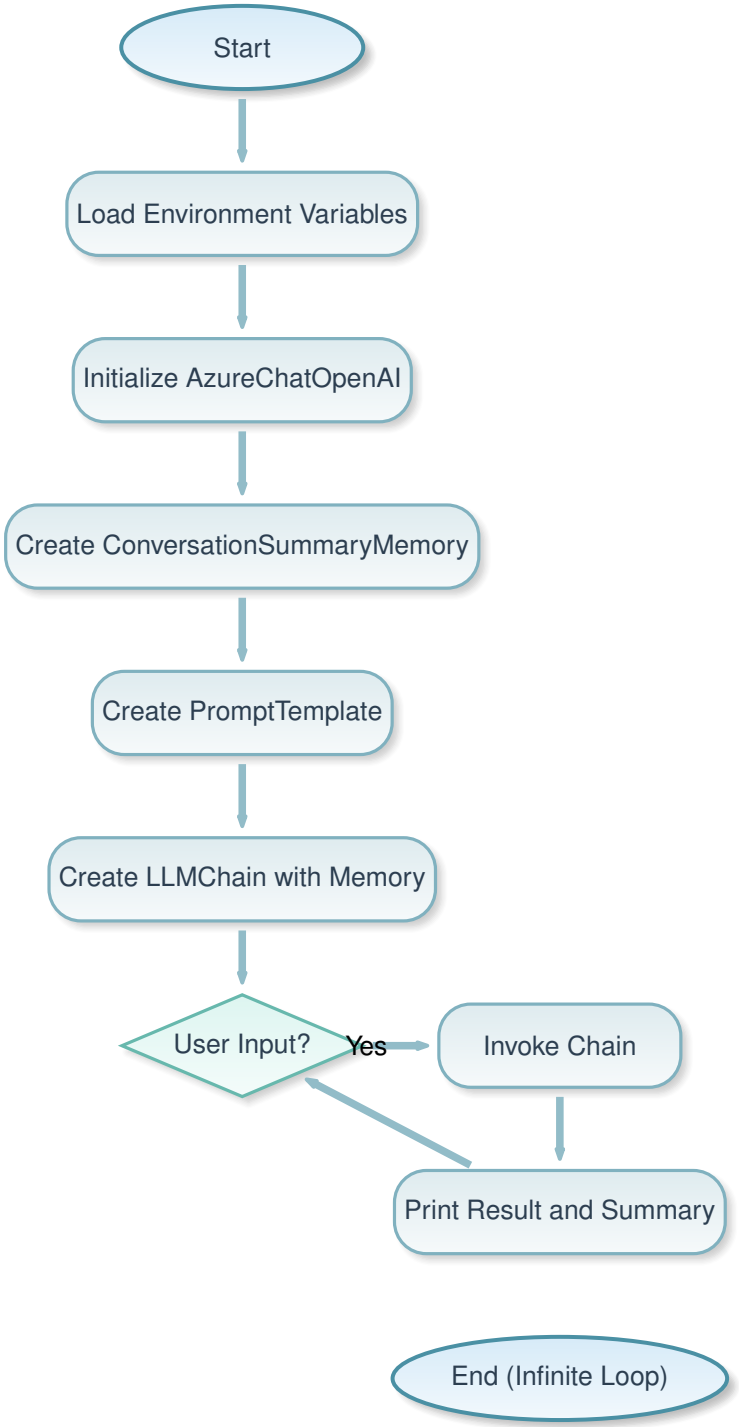
```python
from langchain.chat_models import AzureChatOpenAI
import os
from dotenv import load_dotenv
load_dotenv()
from langchain.chains import LLMChain
from langchain.prompts import PromptTemplate
from langchain.memory import ConversationSummaryMemory

llm_object = AzureChatOpenAI(
    openai_api_base=os.getenv("AZURE_OPENAI_API_BASE"),
    openai_api_version=os.getenv("AZURE_OPENAI_API_VERSION"),
    openai_api_key=os.getenv("AZURE_OPENAI_API_KEY"),
    deployment_name=os.getenv("AZURE_OPENAI_DEPLOYMENT_NAME"),
    model_name="gpt-4o",
    temperature=0.7,
)

memory = ConversationSummaryMemory(
    memory_key="chat_history",
    return_messages=True,
    llm=llm
)

prompt = PromptTemplate(
    input_variables=["chat_history", "user_input"],
    template="""

   "summary in 50 words technical

    Conversation history:
    {chat_history}
    User: {user_input}
    AI:
    """
)

chain = LLMChain(
    llm=llm_object,
    prompt=prompt,
    memory=memory
)

```

```
while True:
    inp = input("Enter the query: ")
    result = chain.invoke({"user_input": inp})
    print(result['text'])
    print(memory.buffer)
```

31.1 Explanation of the Code

- **What:** This code uses ConversationSummaryMemory to maintain a summarized version of the conversation history, condensing past interactions into summaries. It employs Azure OpenAI in a loop for user queries, printing responses and the current memory summary.
- **Why:** Summary memory prevents context from growing too large by summarizing old parts, maintaining efficiency in long conversations. Useful when full history isn't needed but key points are. Leverages Azure for summarization and response generation.
- **How:**
  - Load env vars.
  - Init AzureChatOpenAI (note: llm in memory should be llm_object).
  - Create ConversationSummaryMemory with llm for summarization.
  - Define prompt with history and input.
  - Create LLMChain with memory.
  - Loop: Input, invoke, print result['text'] and memory.buffer (summary).

31.2 Flowchart



32 Ollama Equivalent Code

```python
from langchain_ollama import OllamaLLM
from langchain_core.prompts import PromptTemplate
from langchain_classic.memory import ConversationSummaryMemory

llm_object = OllamaLLM(model="qwen3:4b", temperature=0.7)

memory = ConversationSummaryMemory(
    memory_key="chat_history",
    return_messages=True,
    llm=llm_object
)

prompt = PromptTemplate(
    input_variables=["chat_history", "user_input"],
    template="""

   "summary in 50 words technical

    Conversation history:
    {chat_history}
    User: {user_input}
    AI:
    """
)

# Using pipe operator instead of deprecated LLMChain
chain = prompt | llm_object

while True:
    inp = input("Enter the query: ")
    result = chain.invoke({"user_input": inp, "chat_history": memory.
    ↪ buffer})
    print(result)
    memory.save_context({"input": inp}, {"output": result})
```

32.1 Diff Explanation

— Removed Code

```
- from langchain.chat_models import AzureChatOpenAI
- import os
- from dotenv import load_dotenv
- load_dotenv()
- from langchain.chains import LLMChain
- from langchain.prompts import PromptTemplate
- from langchain.memory import ConversationSummaryMemory
- llm_object = AzureChatOpenAI(...)
- memory = ConversationSummaryMemory(..., llm=llm)
- chain = LLMChain(llm=llm_object, prompt=prompt, memory=memory)
- result = chain.invoke({"user_input": inp})
- print(result['text'])
- print(memory.buffer)
```

+++ Added Code

```
+ from langchain_ollama import OllamaLLM
+ from langchain_core.prompts import PromptTemplate
+ from langchain_classic.memory import ConversationSummaryMemory
+ llm_object = OllamaLLM(model="qwen3:4b", temperature=0.7)
+ memory = ConversationSummaryMemory(..., llm=llm_object)
+ chain = prompt | llm_object
+ result = chain.invoke({"user_input": inp, "chat_history": memory.buffer})
+ print(result)
+ memory.save_context({"input": inp})
```

32.2 Explanation of the Diff

- **Removed Azure:** Local Ollama.
- **Updated imports:** langchain_core and langchain_classic.
- **Modern chaining:** | operator.
- **Manual save:** For memory update.
- **Why:** Adapts summary memory to Ollama, using the same LLM for summarization.

33 Hugging Face Equivalent Code

```python
from transformers import pipeline

pipe = pipeline("text-generation", model="Qwen/Qwen3-0.6B")

chat_history = []

while True:
    inp = input("Enter the query: ")
    history_str = "\n".join(chat_history)
    prompt = f"""

   "summary in 50 words technical

    Conversation history:
    {history_str}
    User: {inp}
    AI:
    """
    result = pipe(prompt, max_new_tokens=100, temperature=0.7,
    ↪ num_return_sequences=1)
    response = result[0]['generated_text'].split("AI:")[-1].strip() if "AI
    ↪ :" in result[0]['generated_text'] else result[0]['generated_text']
    print(response)
    chat_history.append(f"User: {inp}")
    chat_history.append(f"AI: {response}")
    print(chat_history)
```

33.1 Diff Explanation

— Removed Code

```
- from langchain.chat_models import AzureChatOpenAI
- ... (all LangChain)
- memory = ConversationSummaryMemory(...)
- chain = LLMChain(...)
- result = chain.invoke({"user_input": inp})
- print(result['text'])
- print(memory.buffer)
```

+++ Added Code

```
+ from transformers import pipeline
+ pipe = pipeline("text-generation", model="Qwen/Qwen3-0.6B")
+ chat_history = []
```

```
+ history_str = "\n".join(chat_history)
+ prompt = f"..." (manual)
+ result = pipe(prompt, max_new_tokens=100, temperature=0.7, num_return_sequences=1)
+ response = result[0]['generated_text'].split("AI:")[-1].strip() ...
+ print(response)
+ chat_history.append(f"User: {inp}"
+ chat_history.append(f"AI: {response}"
+ print(chat_history)
```

33.2 Explanation of the Diff

- **Removed summary memory:** No summarization, full history.
- **Manual history:** List-based tracking.
- **Why:** Hugging Face lacks built-in summary memory, so simplified to buffer-like behavior.

34 Original Code: code9C.py

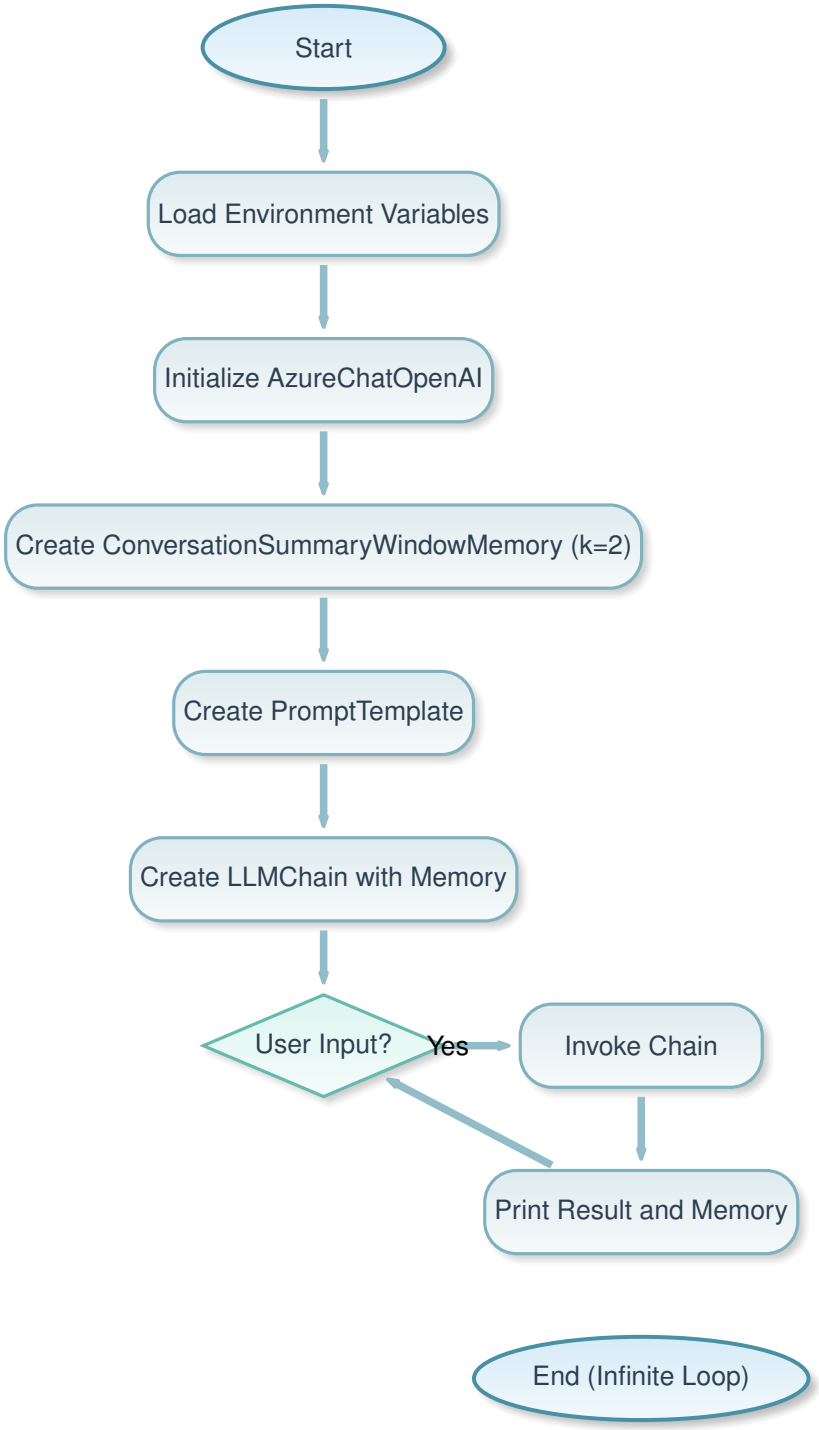
```python
from langchain.chat_models import AzureChatOpenAI
import os
from dotenv import load_dotenv
load_dotenv()
from langchain.chains import LLMChain
from langchain.prompts import PromptTemplate
from langchain.memory import ConversationSummaryWindowMemory

llm_object = AzureChatOpenAI(
    openai_api_base=os.getenv("AZURE_OPENAI_API_BASE"),
    openai_api_version=os.getenv("AZURE_OPENAI_API_VERSION"),
    openai_api_key=os.getenv("AZURE_OPENAI_API_KEY"),
    deployment_name=os.getenv("AZURE_OPENAI_DEPLOYMENT_NAME"),
    model_name="gpt-4o",
    temperature=0.7,
)

memory = ConversationSummaryWindowMemory(
    memory_key="chat_history",
    return_messages=True,
    llm=llm,
    k=2
)

prompt = PromptTemplate(
    input_variables=["chat_history", "user_input"],
    template="""

   "summary in 50 words technical

    Conversation history:
    {chat_history}
    User: {user_input}
    AI:
    """
)

chain = LLMChain(
    llm=llm_object,
    prompt=prompt,
    memory=memory
)
```

```
43
44 while True:
45     inp = input("Enter the query: ")
46     result = chain.invoke({"user_input": inp})
47     print(result['text'])
48     print(memory.buffer)
```

34.1 Explanation of the Code

- **What:** This code employs ConversationSummaryWindowMemory, which summarizes the conversation but keeps the last k interactions in full. It uses Azure OpenAI in a loop, printing responses and the summarized/windowed memory.
- **Why:** Combines summarization for old parts with full retention of recent interactions, balancing efficiency and recency. Ideal for conversations needing recent context but compressed history. Uses Azure for both generation and summarization.
- **How:**
  - Load env vars.
  - Init AzureChatOpenAI.
  - Create ConversationSummaryWindowMemory with llm (should be llm_object) and k=2.
  - Define prompt.
  - Create LLMChain.
  - Loop: Input, invoke, print result['text'] and memory.buffer.

34.2 Flowchart



35 Ollama Equivalent Code

```python
from langchain_ollama import OllamaLLM
from langchain_core.prompts import PromptTemplate
from langchain_classic.memory import ConversationSummaryBufferMemory

llm_object = OllamaLLM(model="qwen3:4b", temperature=0.7)

memory = ConversationSummaryBufferMemory(
    memory_key="chat_history",
    return_messages=True,
    llm=llm_object,
)

prompt = PromptTemplate(
    input_variables=["chat_history", "user_input"],
    template="""

   "summary in 50 words technical

    Conversation history:
    {chat_history}
    User: {user_input}
    AI:
    """
)

# Using pipe operator instead of deprecated LLMChain
chain = prompt | llm_object

while True:
    inp = input("Enter the query: ")
    result = chain.invoke({"user_input": inp, "chat_history": memory.
    ↪ buffer})
    print(result)
    memory.save_context({"input": inp}, {"output": result})
```

35.1 Diff Explanation

— Removed Code

```
- from langchain.chat_models import AzureChatOpenAI
- import os
- from dotenv import load_dotenv
- load_dotenv()
- from langchain.chains import LLMChain
- from langchain.prompts import PromptTemplate
- from langchain.memory import ConversationSummaryWindowMemory
- llm_object = AzureChatOpenAI(...)
- memory = ConversationSummaryWindowMemory(..., llm=llm, k=2)
- chain = LLMChain(llm=llm_object, prompt=prompt, memory=memory)
- result = chain.invoke({"user_input": inp})
- print(result['text'])
- print(memory.buffer)
```

+++ Added Code

```
+ from langchain_ollama import OllamaLLM
+ from langchain_core.prompts import PromptTemplate
+ from langchain_classic.memory import ConversationSummaryBufferMemory
+ llm_object = OllamaLLM(model="qwen3:4b", temperature=0.7)
+ memory = ConversationSummaryBufferMemory(..., llm=llm_object)
+ chain = prompt | llm_object
+ result = chain.invoke({"user_input": inp, "chat_history": memory.buffer})
+ print(result)
+ memory.save_context({"input": inp})
```

35.2 Explanation of the Diff

- **Removed Azure:** Local Ollama.
- **Changed memory type:** ConversationSummaryBufferMemory instead of Window, as Window may not be available.
- **Modern chaining:** | operator.
- **Why:** Adapts to available memory types in langchain_classic, using buffer with summarization.

36 Hugging Face Equivalent Code

```python
from transformers import pipeline

pipe = pipeline("text-generation", model="Qwen/Qwen3-0.6B")

chat_history = []

while True:
    inp = input("Enter the query: ")
    # Approximate: keep last 4 messages
    if len(chat_history) > 4:
        chat_history = chat_history[-4:]
    history_str = "\n".join(chat_history)
    prompt = f"""

   "summary in 50 words technical

    Conversation history:
    {history_str}
    User: {inp}
    AI:
    """
    result = pipe(prompt, max_new_tokens=100, temperature=0.7,
    ↪ num_return_sequences=1)
    response = result[0]['generated_text'].split("AI:")[-1].strip() if "AI
    ↪ :" in result[0]['generated_text'] else result[0]['generated_text']
    print(response)
    chat_history.append(f"User: {inp}")
    chat_history.append(f"AI: {response}")
    print(chat_history)
```

36.1 Diff Explanation

— Removed Code

```
- from langchain.chat_models import AzureChatOpenAI
- ... (all LangChain)
- memory = ConversationSummaryWindowMemory(...)
- chain = LLMChain(...)
- result = chain.invoke({"user_input": inp})
```

```
- print(result['text'])
- print(memory.buffer)
```

+++ Added Code

```
+ from transformers import pipeline
+ pipe = pipeline("text-generation", model="Qwen/Qwen3-0.6B")
+ chat_history = []
+ if len(chat_history) > 4: chat_history = chat_history[-4:]
+ history_str = "\n".join(chat_history)
+ prompt = f"..." (manual)
+ result = pipe(prompt, max_new_tokens=100, temperature=0.7, num_return_sequences=1)
+ response = result[0]['generated_text'].split("AI:")[-1].strip() ...
+ print(response)
+ chat_history.append(f"User: {inp}"
+ chat_history.append(f"AI: {response}"
+ print(chat_history)
```

36.2 Explanation of the Diff

- **Removed summary window memory:** Manual windowing without summarization.
- **Why:** Hugging Face doesn't have built-in summary memory, so approximated with buffer window.

37 Original Code: code10.py

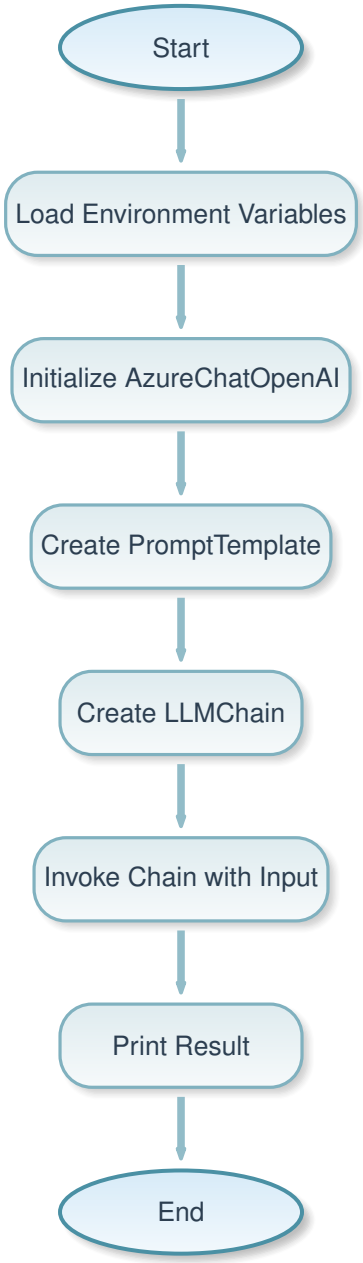
```python
#component 1 LLM
from langchain.chat_models import AzureChatOpenAI
import os
from dotenv import load_dotenv
load_dotenv()
llm = AzureChatOpenAI(
    openai_api_base=os.getenv("AZURE_OPENAI_API_BASE"),
    openai_api_version=os.getenv("AZURE_OPENAI_API_VERSION"),
    openai_api_key=os.getenv("AZURE_OPENAI_API_KEY"),
    deployment_name=os.getenv("AZURE_OPENAI_DEPLOYMENT_NAME"),
    model_name="gpt-4o",
    temperature=0.7,
)


from langchain.chains import LLMChain

#compenent 2 PROMPT
from langchain.prompts import PromptTemplate
prompt=PromptTemplate(
    template="what is the capital of {input}"
)

pipeline=LLMChain(
    llm=llm,
    prompt=prompt
)


result=pipeline.invoke(input="china")

print(result['text'])
```

37.1 Explanation of the Code

- **What:** This code creates a simple pipeline using LLMChain that combines a prompt template with Azure OpenAI LLM, invokes it with "china" as input, and prints the response.
- **Why:** Demonstrates basic chaining of prompt and LLM in LangChain for templated queries. Uses Azure for cloud-based AI, with env vars for security.

- **How:**
  - Load env vars.
  - Init AzureChatOpenAI.
  - Create PromptTemplate with placeholder.
  - Create LLMChain with llm and prompt.
  - Invoke with dict input.
  - Print result['text'].

37.2 Flowchart



38 Ollama Equivalent Code

```
from langchain_ollama import OllamaLLM
from langchain_core.prompts import PromptTemplate
```

```

llm = OllamaLLM(model="qwen3:4b", temperature=0.7)

prompt = PromptTemplate(
    template="what is the capital of {input}"
)

# Using pipe operator instead of deprecated LLMChain
pipeline = prompt | llm

result = pipeline.invoke({"input": "china"})

print(result)
```

38.1 Diff Explanation

— Removed Code

```
- from langchain.chat_models import AzureChatOpenAI
- import os
- from dotenv import load_dotenv
- load_dotenv()
- llm = AzureChatOpenAI(...)
- from langchain.chains import LLMChain
- from langchain.prompts import PromptTemplate
- prompt=PromptTemplate(...)
- pipeline=LLMChain(llm=llm, prompt=prompt)
- result=pipeline.invoke(input="china")
- print(result['text'])
```

+++ Added Code

```
+ from langchain_ollama import OllamaLLM
+ from langchain_core.prompts import PromptTemplate
+ llm = OllamaLLM(model="qwen3:4b", temperature=0.7)
+ prompt = PromptTemplate(...)
+ pipeline = prompt | llm
+ result = pipeline.invoke({"input": "china"})
+ print(result)
```

38.2 Explanation of the Diff

- **Removed Azure setup:** Local Ollama.
- **Updated imports:** langchain_core.

- **Modern chaining:** | instead of LLMChain.
- **Why:** Simplifies to local execution with updated LangChain syntax.

39 Hugging Face Equivalent Code

```python
from transformers import pipeline

pipe = pipeline("text-generation", model="Qwen/Qwen3-0.6B")

prompt_template = "what is the capital of {input}"

input_value = "china"
prompt = prompt_template.format(input=input_value)

result = pipe(prompt, max_new_tokens=50, temperature=0.7,
    ↪ num_return_sequences=1)

print(result[0]['generated_text'])
```

39.1 Diff Explanation

— Removed Code

```
- from langchain.chat_models import AzureChatOpenAI
- ... (all LangChain)
- pipeline=LLMChain(...)
- result=pipeline.invoke(input="china")
- print(result['text'])
```

+++ Added Code

```
+ from transformers import pipeline
+ pipe = pipeline("text-generation", model="Qwen/Qwen3-0.6B")
+ prompt_template = "what is the capital of {input}"
+ input_value = "china"
+ prompt = prompt_template.format(input=input_value)
+ result = pipe(prompt, max_new_tokens=50, temperature=0.7, num_return_sequences=1)
+ print(result[0]['generated_text'])
```

39.2 Explanation of the Diff

- **Removed LangChain:** Manual template formatting.
- **Direct pipeline:** No chaining abstraction.
- **Why:** Hugging Face requires explicit prompt construction.

40 Original Code: code11.py

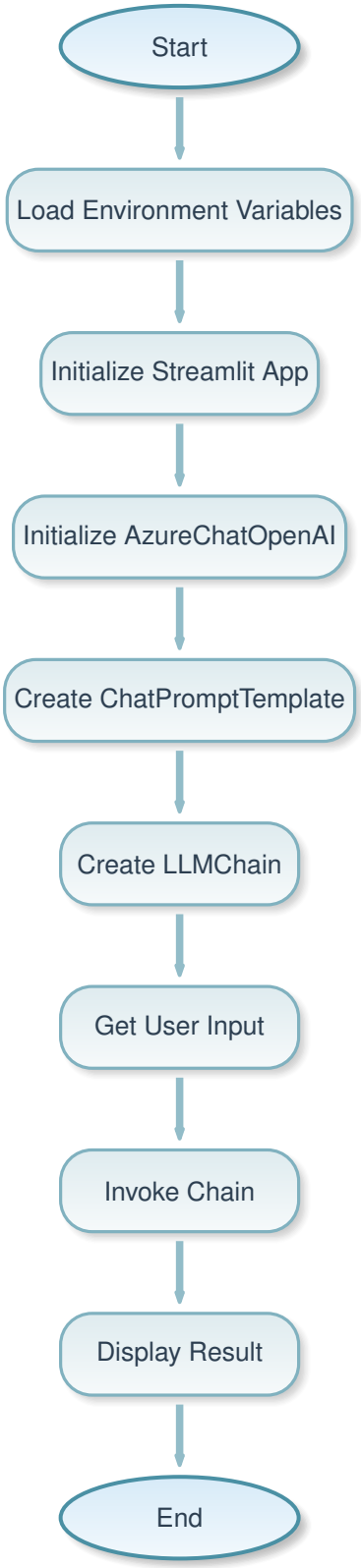
```python
# Component -1
from langchain.chat_models import AzureChatOpenAI
import os
from dotenv import load_dotenv
load_dotenv()
import streamlit as st
st.title("MY FIRST CHATBOT")
my_llm = AzureChatOpenAI(
    openai_api_base=os.getenv("AZURE_OPENAI_API_BASE"),
    openai_api_version=os.getenv("AZURE_OPENAI_API_VERSION"),
    openai_api_key=os.getenv("AZURE_OPENAI_API_KEY"),
    deployment_name=os.getenv("AZURE_OPENAI_DEPLOYMENT_NAME"),
    model_name="gpt-4o",
    temperature=0.7,
)

# Component -2  (ChatPromptTemplate Version)
from langchain.prompts import ChatPromptTemplate

my_prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a helpful assistant."),
    ("human", "{input}")
])

from langchain.chains import LLMChain

my_pipeline = LLMChain(
    llm=my_llm,
    prompt=my_prompt
)
usertext=st.text_input("USER:")
result = my_pipeline.invoke({"input": usertext})
st.write(result["text"])
```

40.1 Explanation of the Code

- **What:** This code builds a Streamlit web app for a chatbot, using ChatPromptTemplate with system and human messages, combined with Azure OpenAI in an LLMChain, taking user input and displaying the AI response.

- **Why:** Demonstrates creating a simple web-based chatbot with LangChain's chat prompt templates for structured conversations. Uses Streamlit for UI and Azure for AI.
- **How:**
  - Load env vars.
  - Init Streamlit title.
  - Init AzureChatOpenAI.
  - Create ChatPromptTemplate with messages.
  - Create LLMChain.
  - Get user input via st.text_input.
  - Invoke chain and display result["text"].

40.2 Flowchart



41 Ollama Equivalent Code

```python
# Component -1
from langchain_ollama import OllamaLLM
import streamlit as st
st.title("MY FIRST CHATBOT")
my_llm = OllamaLLM(model="qwen3:4b", temperature=0.7)

# Component -2  (ChatPromptTemplate Version)
from langchain_core.prompts import ChatPromptTemplate

my_prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a helpful assistant."),
    ("human", "{input}")
])

# Using pipe operator instead of deprecated LLMChain
my_pipeline = my_prompt | my_llm
usertext=st.text_input("USER:")
result = my_pipeline.invoke({"input": usertext})
st.write(result["text"])
```

41.1 Diff Explanation

— Removed Code

```
- from langchain.chat_models import AzureChatOpenAI
- import os
- from dotenv import load_dotenv
- load_dotenv()
- my_llm = AzureChatOpenAI(...)
- from langchain.prompts import ChatPromptTemplate
- from langchain.chains import LLMChain
- my_pipeline = LLMChain(llm=my_llm, prompt=my_prompt)
```

+++ Added Code

```
+ from langchain_ollama import OllamaLLM
+ my_llm = OllamaLLM(model="qwen3:4b", temperature=0.7)
+ from langchain_core.prompts import ChatPromptTemplate
+ my_pipeline = my_prompt | my_llm
```

41.2 Explanation of the Diff

- **Removed Azure:** Local Ollama.
- **Updated imports:** langchain_core.
- **Modern chaining:** | operator.
- **Why:** Adapts to local model with updated syntax.

42 Hugging Face Equivalent Code

```python
from transformers import pipeline
import streamlit as st
st.title("MY FIRST CHATBOT")

generator = pipeline("text-generation", model="gpt2")

usertext = st.text_input("USER:")
if usertext:
    prompt = f"You are a helpful assistant. {usertext}"
    result = generator(prompt, max_length=50, num_return_sequences=1)
    st.write(result[0]['generated_text'])
```

42.1 Diff Explanation

— Removed Code

```
- from langchain.chat_models import AzureChatOpenAI
- ... (all LangChain)
- my_pipeline = LLMChain(...)
- result = my_pipeline.invoke({"input": usertext})
- st.write(result["text"])
```

+++ Added Code

```
+ from transformers import pipeline
+ generator = pipeline("text-generation", model="gpt2")
+ usertext = st.text_input("USER:")
+ if usertext:
+     prompt = f"You are a helpful assistant. {usertext}"
```

```
+     result = generator(prompt, max_length=50, num_return_sequences=1)
+     st.write(result[0]['generated_text'])
```

42.2 Explanation of the Diff

- **Removed LangChain:** Manual prompt construction.
- **Changed model:** Uses gpt2 instead of Qwen.
- **Why:** Hugging Face requires direct pipeline usage and prompt engineering.

43 Original Code: code12.py

```
# In a RAG pipeline you must:

# 1. Collect documents (PDFs, text files, DOCX, website pages, database
    ↪ rows, etc.)


# 2. Load them into your program


# 3. Split them into chunks


# 4. Convert them into embeddings


# 5. Store them in a vector database


# Here we focus on Step 2 -- Loading documents.


# ---

# Most Common Ways to Load Documents for RAG

# Here are the most frequently used loaders in Python (LangChain):


# ---

# Load PDFs

# Using LangChain's PyPDFLoader

from langchain.document_loaders import PyPDFLoader

loader = PyPDFLoader("sample.pdf")
documents = loader.load()

print(documents)

```

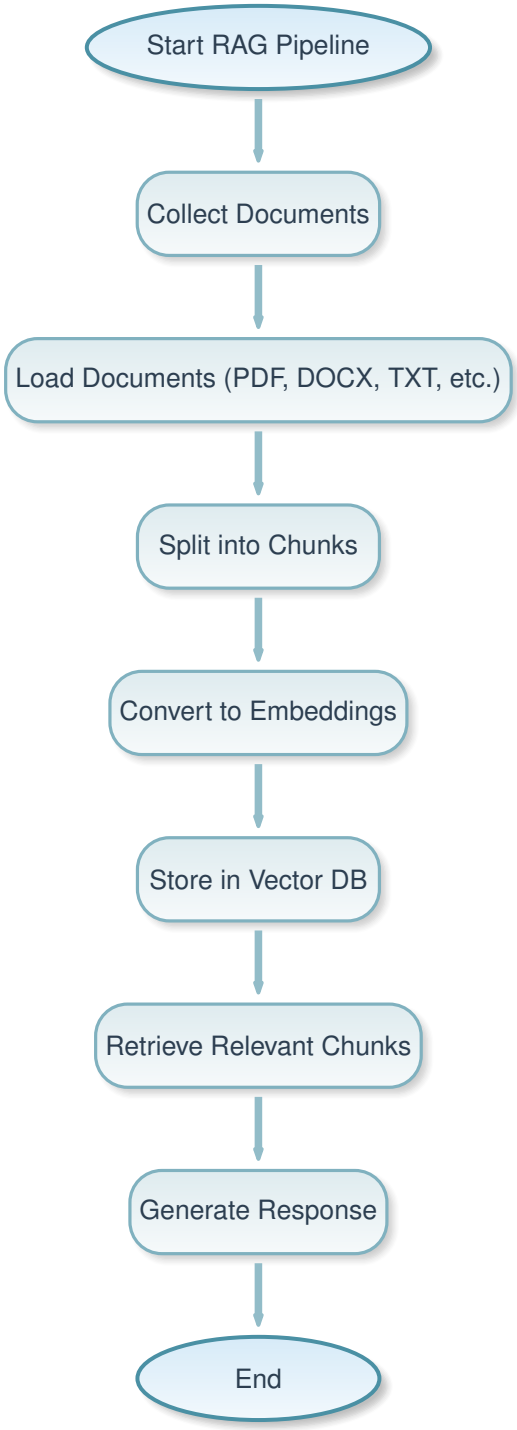
```

# Load DOCX / Word files
from langchain.document_loaders import Docx2txtLoader
loader = Docx2txtLoader("sample.docx")
documents = loader.load()


# Load Text Files (.txt)
from langchain.document_loaders import TextLoader
loader = TextLoader("notes.txt")
documents = loader.load()

# Load Multiple Documents in a Folder
from langchain.document_loaders import DirectoryLoader
loader = DirectoryLoader(
    "data/",
    glob="*/.pdf",
    show_progress=True
)
documents = loader.load()

Load Website Pages (HTML)
from langchain.document_loaders import WebBaseLoader
loader = WebBaseLoader("http://shooliniuniversity.com/")
documents = loader.load()

# Load Markdown Files (.md)
from langchain.document_loaders import UnstructuredMarkdownLoader
loader = UnstructuredMarkdownLoader("readme.md")
```

```
documents = loader.load()


# Load CSV Files

from langchain.document_loaders.csv_loader import CSVLoader

loader = CSVLoader("data.csv")
documents = loader.load()
```

43.1 Explanation of the Code

- **What:** This code provides examples of various document loaders in LangChain for RAG (Retrieval-Augmented Generation) systems, including loaders for PDFs, DOCX, text files, directories, websites, Markdown, and CSV files.
- **Why:** Document loading is a crucial step in RAG pipelines to ingest diverse data sources into the system for later chunking, embedding, and retrieval. LangChain provides unified interfaces for different file types to simplify data ingestion.
- **How:**
  - Import specific loader classes from langchain.document_loaders.
  - Instantiate loader with file path or URL.
  - Call load() method to get documents (list of Document objects).
  - Print or process the loaded documents.

43.2 Flowchart



44 Ollama Equivalent Code

```
# In a RAG pipeline you must:

# 1. Collect documents (PDFs, text files, DOCX, website pages, database rows, etc.)


# 2. Load them into your program


# 3. Split them into chunks


# 4. Convert them into embeddings


# 5. Store them in a vector database


# Here we focus on Step 2 -- Loading documents.


# ---

# Most Common Ways to Load Documents for RAG

# Here are the most frequently used loaders in Python (LangChain):


# ---

# Load PDFs

# Using LangChain's PyPDFLoader

from langchain_community.document_loaders import PyPDFLoader

loader = PyPDFLoader("sample.pdf")
documents = loader.load()

print(documents)

```

```python


# Load DOCX / Word files

from langchain_community.document_loaders import Docx2txtLoader

loader = Docx2txtLoader("sample.docx")
documents = loader.load()



# Load Text Files (.txt)

from langchain_community.document_loaders import TextLoader

loader = TextLoader("notes.txt")
documents = loader.load()


# Load Multiple Documents in a Folder

from langchain_community.document_loaders import DirectoryLoader

loader = DirectoryLoader(
    "data/",
    glob="*/.pdf",
    show_progress=True
)

documents = loader.load()


# Load Website Pages (HTML)

from langchain_community.document_loaders import WebBaseLoader

loader = WebBaseLoader("http://shooliniuniversity.com/")
documents = loader.load()


# Load Markdown Files (.md)

from langchain_community.document_loaders import
    ↪ UnstructuredMarkdownLoader

loader = UnstructuredMarkdownLoader("readme.md")
```

```
documents = loader.load()


# Load CSV Files

from langchain_community.document_loaders.csv_loader import CSVLoader

loader = CSVLoader("data.csv")
documents = loader.load()

loader = CSVLoader("data.csv")
documents = loader.load()
```

44.1 Diff Explanation

— Removed Code

```
- from langchain.document_loaders import PyPDFLoader
- from langchain.document_loaders import Docx2txtLoader
- from langchain.document_loaders import TextLoader
- from langchain.document_loaders import DirectoryLoader
- from langchain.document_loaders import WebBaseLoader
- from langchain.document_loaders import UnstructuredMarkdownLoader
- from langchain.document_loaders.csv_loader import CSVLoader
```

+++ Added Code

```
+ from langchain_community.document_loaders import PyPDFLoader
+ from langchain_community.document_loaders import Docx2txtLoader
+ from langchain_community.document_loaders import TextLoader
+ from langchain_community.document_loaders import DirectoryLoader
+ from langchain_community.document_loaders import WebBaseLoader
+ from langchain_community.document_loaders import UnstructuredMarkdownLoader
+ from langchain_community.document_loaders.csv_loader import CSVLoader
```

44.2 Explanation of the Diff

- **Changed import paths:** From `langchain.document_loaders` to `langchain_community.document_loaders`.
- **Why:** In newer LangChain versions, community-contributed loaders are moved to the `langchain_community` package for better organization.

45 Hugging Face Equivalent Code

```
# In a RAG pipeline you must:

# 1. Collect documents (PDFs, text files, DOCX, website pages, database
    ↪ rows, etc.)


# 2. Load them into your program


# 3. Split them into chunks


# 4. Convert them into embeddings


# 5. Store them in a vector database


# Here we focus on Step 2 -- Loading documents.


# ---

# Most Common Ways to Load Documents for RAG

# Here are the most frequently used loaders in Python (LangChain):


# ---

# Load PDFs

# Using LangChain's PyPDFLoader

from langchain_community.document_loaders import PyPDFLoader

loader = PyPDFLoader("sample.pdf")
documents = loader.load()

print(documents)

```

```python


# Load DOCX / Word files

from langchain_community.document_loaders import Docx2txtLoader

loader = Docx2txtLoader("sample.docx")
documents = loader.load()



# Load Text Files (.txt)

from langchain_community.document_loaders import TextLoader

loader = TextLoader("notes.txt")
documents = loader.load()


# Load Multiple Documents in a Folder

from langchain_community.document_loaders import DirectoryLoader

loader = DirectoryLoader(
    "data/",
    glob="*/.pdf",
    show_progress=True
)

documents = loader.load()


# Load Website Pages (HTML)

from langchain_community.document_loaders import WebBaseLoader

loader = WebBaseLoader("http://shooliniuniversity.com/")
documents = loader.load()


# Load Markdown Files (.md)

from langchain_community.document_loaders import
    ↪ UnstructuredMarkdownLoader

loader = UnstructuredMarkdownLoader("readme.md")
```

```
documents = loader.load()


# Load CSV Files

from langchain_community.document_loaders.csv_loader import CSVLoader

loader = CSVLoader("data.csv")
documents = loader.load()

loader = CSVLoader("data.csv")
documents = loader.load()
```

45.1 Diff Explanation

Same as Ollama: Changed import paths to langchain_community.

45.2 Explanation of the Diff

Same as Ollama.

46 Original Code: code13.py

```python
from langchain.document_loaders import TextLoader


path='ipl.txt'

def load_doc(path):
  if path.endswith('.txt'):
    loader=TextLoader(path)
  documents=loader.load()
  return documents

documents=load_doc(path)
print(documents)


# #Now we do chunking as already told model have limit and answer also
    ↪ become accurate

# #1. CharacterTextSplitter

from langchain.text_splitter import CharacterTextSplitter


chunking=CharacterTextSplitter(chunk_size=70,chunk_overlap=39)


docs=chunking.split_documents(documents)

print(len(docs))


for i in range(len(docs)):
    print(docs[i])
    print("----------------------------------------")
```

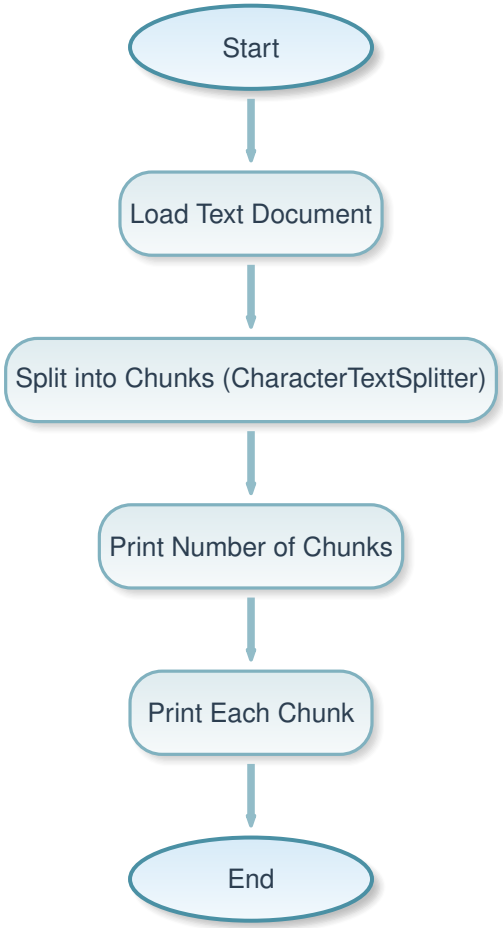
46.1 Explanation of the Code

- **What:** This code loads a text file using TextLoader, then splits the document into chunks using CharacterTextSplitter with specified chunk size and overlap, and prints the chunks.
- **Why:** In RAG systems, large documents must be split into manageable chunks to fit model

context limits and improve retrieval accuracy. Character-based splitting is simple and preserves some context via overlap.

- **How:**
  - Load text file with TextLoader.
  - Create CharacterTextSplitter with chunk_size=70 and chunk_overlap=39.
  - Split documents into chunks.
  - Print number of chunks and each chunk.

46.2 Flowchart



47 Ollama Equivalent Code

```python
from langchain_community.document_loaders import TextLoader


path='ipl.txt'

def load_doc(path):
  if path.endswith('.txt'):
    loader=TextLoader(path)
  documents=loader.load()
  return documents

documents=load_doc(path)
print(documents)


# #Now we do chunking as already told model have limit and answer also become accurate

# #1. CharacterTextSplitter

from langchain_text_splitters import CharacterTextSplitter


chunking=CharacterTextSplitter(chunk_size=70,chunk_overlap=39)


docs=chunking.split_documents(documents)

print(len(docs))


for i in range(len(docs)):
    print(docs[i])
    print("----------------------------------------")
```

47.1 Diff Explanation

— Removed Code

```
- from langchain.document_loaders import TextLoader
- from langchain.text_splitter import CharacterTextSplitter
```

+++ Added Code

```
+ from langchain_community.document_loaders import TextLoader
+ from langchain_text_splitters import CharacterTextSplitter
```

47.2 Explanation of the Diff

- **Changed import paths:** Loaders to langchain_community, splitters to langchain_text_splitters.
- **Why:** Modularization in newer LangChain versions.

48 Hugging Face Equivalent Code

Same as Ollama.

48.1 Diff Explanation

Same as Ollama.

49 Original Code: code13A.py

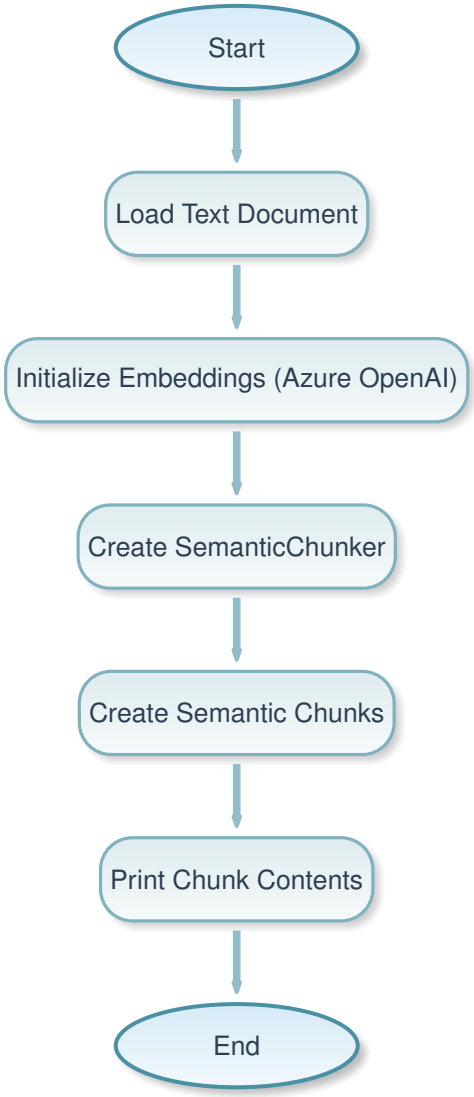
```python
from langchain.document_loaders import TextLoader


path='ipl.txt'

def load_doc(path):
  if path.endswith('.txt'):
    loader=TextLoader(path)
  documents=loader.load()
  return documents

documents=load_doc(path)
print(documents)


# #Now we do chunking as already told model have limit and answer also become accurate


import os
from langchain_openai import AzureOpenAIEmbeddings


AZURE_OPENAI_API_KEY = ""  #Put your own API key
AZURE_OPENAI_ENDPOINT = "https://chatgpt-key.openai.azure.com/"
AZURE_OPENAI_DEPLOYMENT_NAME = "gpt-4o-2"
AZURE_OPENAI_API_VERSION = "2025-01-01-preview"
AZURE_OPENAI_EMBEDDING_DEPLOYMENT_NAME = "text-embedding-3-large" # Replace with your embedding deployment name
AZURE_OPENAI_EMBEDDING_MODEL_NAME = "text-embedding-3-large" # Replace with your embedding model name


embeddings = AzureOpenAIEmbeddings(
    deployment=AZURE_OPENAI_EMBEDDING_DEPLOYMENT_NAME,
    model=AZURE_OPENAI_EMBEDDING_MODEL_NAME,
    api_key=AZURE_OPENAI_API_KEY,
    azure_endpoint=AZURE_OPENAI_ENDPOINT,
    api_version=AZURE_OPENAI_API_VERSION,
)


```

```
from langchain_experimental.text_splitter import SemanticChunker
from langchain_openai import OpenAIEmbeddings


chunker = SemanticChunker(embeddings)
chunks = chunker.create_documents([documents])

for c in chunks:
    print(c.page_content)
```

49.1 Explanation of the Code

- **What:** This code loads a text file, initializes Azure OpenAI embeddings, and uses SemanticChunker to split the document into semantically coherent chunks based on embeddings.
- **Why:** Semantic chunking groups text by meaning rather than fixed characters, leading to better retrieval in RAG as chunks maintain topical coherence. Uses embeddings to determine split points.
- **How:**
  - Load text document.
  - Set up Azure OpenAI embeddings with API details.
  - Create SemanticChunker with embeddings.
  - Generate chunks from documents.
  - Print each chunk's content.

49.2 Flowchart



50 Ollama Equivalent Code

```
from langchain_community.document_loaders import TextLoader


path='ipl.txt'

def load_doc(path):
  if path.endswith('.txt'):
```

```
    loader=TextLoader(path)
  documents=loader.load()
  return documents

documents=load_doc(path)
print(documents)


# #Now we do chunking as already told model have limit and answer also
    ↪ become accurate


from langchain_community.embeddings import HuggingFaceEmbeddings


embeddings = HuggingFaceEmbeddings(model_name="BAAI/bge-m3")


from langchain_experimental.text_splitter import SemanticChunker


chunker = SemanticChunker(embeddings)
chunks = chunker.create_documents([documents])

for c in chunks:
    print(c.page_content)
```

50.1 Diff Explanation

— Removed Code

```
- from langchain.document_loaders import TextLoader
- import os
- from langchain_openai import AzureOpenAIEmbeddings
- AZURE_OPENAI_API_KEY = "" ... (Azure setup)
- embeddings = AzureOpenAIEmbeddings(...)
- from langchain_experimental.text_splitter import SemanticChunker
- from langchain_openai import OpenAIEmbeddings
```

+++ Added Code

```
+ from langchain_community.document_loaders import TextLoader
+ from langchain_community.embeddings import HuggingFaceEmbeddings
+ embeddings = HuggingFaceEmbeddings(model_name="BAAI/bge-m3")
```

```
+ from langchain_experimental.text_splitter import SemanticChunker
```

50.2 Explanation of the Diff

- **Removed Azure embeddings:** Replaced with local HuggingFace embeddings.
- **Changed imports:** To langchain_community.
- **Why:** Adapts to local embeddings for Ollama, avoiding cloud API costs.

51 Hugging Face Equivalent Code

Same as Ollama.

51.1 Diff Explanation

Same as Ollama.

52 Original Code: code14.py

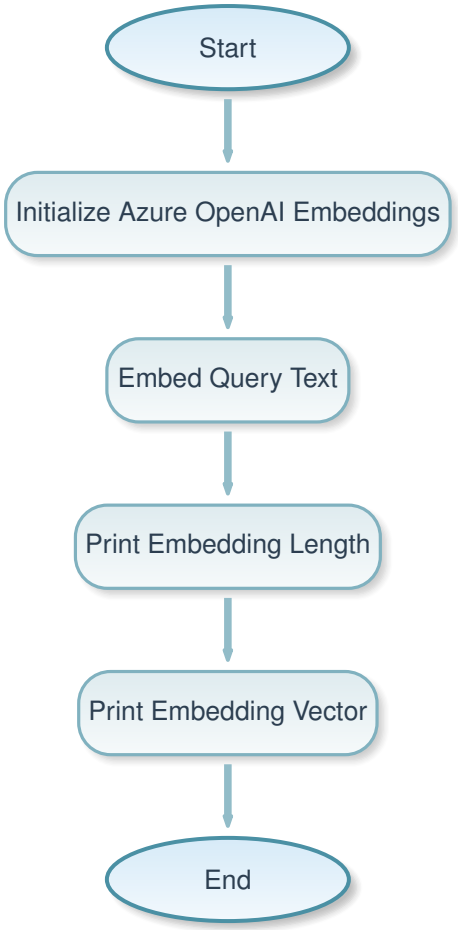
```python
import os
from langchain_openai import AzureOpenAIEmbeddings

AZURE_OPENAI_API_KEY = ""  #Put your own API key
AZURE_OPENAI_ENDPOINT = "https://chatgpt-key.openai.azure.com/"
AZURE_OPENAI_DEPLOYMENT_NAME = "gpt-4o-2"
AZURE_OPENAI_API_VERSION = "2025-01-01-preview"
AZURE_OPENAI_EMBEDDING_DEPLOYMENT_NAME = "text-embedding-3-large" # Replace with your embedding deployment name
AZURE_OPENAI_EMBEDDING_MODEL_NAME = "text-embedding-3-large" # Replace with your embedding model name


embeddings = AzureOpenAIEmbeddings(
    deployment=AZURE_OPENAI_EMBEDDING_DEPLOYMENT_NAME,
    model=AZURE_OPENAI_EMBEDDING_MODEL_NAME,
    api_key=AZURE_OPENAI_API_KEY,
    azure_endpoint=AZURE_OPENAI_ENDPOINT,
    api_version=AZURE_OPENAI_API_VERSION,
)

text = "I love machine learning"


embedding = embeddings.embed_query(text)

print(f"Embedding vector length: {len(embedding)}")
print(embedding)
```

52.1 Explanation of the Code

- **What:** This code initializes Azure OpenAI embeddings and generates an embedding vector for the text "I love machine learning", then prints the vector length and the vector.
- **Why:** Embeddings convert text into numerical vectors for semantic similarity in RAG systems. Azure provides high-quality embeddings via API.
- **How:**
  - Set up Azure OpenAI embeddings with API details.

- Call embed_query on the text.
- Print length and vector.

52.2 Flowchart



53 Ollama Equivalent Code

```
from langchain_community.embeddings import HuggingFaceEmbeddings

embedding_model = HuggingFaceEmbeddings(model_name="BAAI/bge-m3")

text = "I love machine learning"

embedding = embedding_model.embed_query(text)

```

```
print(f"Embedding vector length: {len(embedding)}")
print(embedding)
```

53.1 Diff Explanation

— Removed Code

```
- import os
- from langchain_openai import AzureOpenAIEmbeddings
- AZURE_OPENAI_API_KEY = "" ... (Azure setup)
- embeddings = AzureOpenAIEmbeddings(...)
```

+++ Added Code

```
+ from langchain_community.embeddings import HuggingFaceEmbeddings
+ embedding_model = HuggingFaceEmbeddings(model_name="BAAI/bge-m3")
```

53.2 Explanation of the Diff

- **Removed Azure setup:** Local embeddings.
- **Changed to HuggingFace:** Uses BAAI/bge-m3 model.
- **Why:** Local execution without API keys.

54 Hugging Face Equivalent Code

```
from FlagEmbedding import BGEM3FlagModel

model=BGEM3FlagModel('BAAI/bge-m3',use_fp16=True)

sentence1="I love machine learning"

embeddings1=model.encode(sentence1,
                         batch_size=12,
                         max_length=8192,)['dense_vecs']

print(embeddings1)
```

54.1 Diff Explanation

— Removed Code

```
- from langchain_community.embeddings import HuggingFaceEmbeddings
- embedding_model = HuggingFaceEmbeddings(model_name="BAAI/bge-m3")
- embedding = embedding_model.embed_query(text)
- print(f"Embedding vector length: {len(embedding)}")
- print(embedding)
```

+++ Added Code

```
+ from FlagEmbedding import BGEM3FlagModel
+ model=BGEM3FlagModel('BAAI/bge-m3',use_fp16=True)
+ sentence1="I love machine learning"
+ embeddings1=model.encode(sentence1, batch_size=12, max_length=8192,)['dense_vecs']
+ print(embeddings1)
```

54.2 Explanation of the Diff

- **Removed LangChain wrapper:** Direct FlagEmbedding usage.
- **Changed method:** encode instead of embed_query.
- **Why:** Direct access to Hugging Face embedding models for more control.

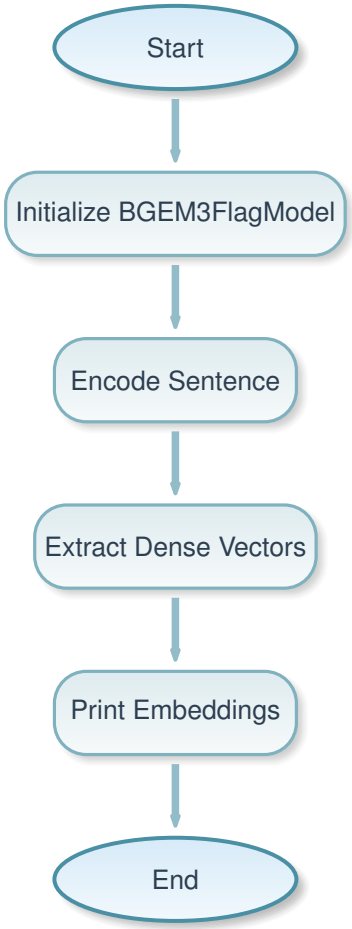
55 Original Code: code14A.py

```python
# !pip install -U FlagEmbedding

from FlagEmbedding import BGEM3FlagModel

model=BGEM3FlagModel('BAAI/bge-m3',use_fp16=True)

sentence1="I am going to school"

embeddings1=model.encode(sentence1,
                         batch_size=12,
                         max_length=8192,)['dense_vecs']

print(embeddings1)
```

55.1 Explanation of the Code

- **What:** This code loads the BGE-M3 embedding model from FlagEmbedding and generates dense embeddings for the sentence "I am going to school", then prints the embedding vector.
- **Why:** Direct use of advanced embedding models for text representation. BGE-M3 is a state-of-the-art model for semantic embeddings.
- **How:**
  - Install FlagEmbedding if needed.
  - Initialize BGEM3FlagModel with model name and fp16.
  - Encode the sentence with parameters.
  - Extract and print dense vectors.

55.2 Flowchart



56 Ollama Equivalent Code

Same as original.

56.1 Diff Explanation

No changes.

56.2 Explanation of the Diff

All versions use the same direct FlagEmbedding approach.

57 Hugging Face Equivalent Code

Same as original.

57.1 Diff Explanation

No changes.

58 Original Code (src/code15.py)

The original code demonstrates computing cosine similarity between two sentences using the BGEM3FlagModel from FlagEmbedding.

```
#cosine similarity


# !pip install -U FlagEmbedding

from FlagEmbedding import BGEM3FlagModel
from sklearn.metrics.pairwise import cosine_similarity

model=BGEM3FlagModel('BAAI/bge-m3',use_fp16=True)

sentence1="I am going to school"

embeddings1=model.encode(sentence1,
                         batch_size=12,
                         max_length=8192,)['dense_vecs']

print(embeddings1)

sentence2="I am going to home"

embeddings2=model.encode(sentence2,
                         batch_size=12,
                         max_length=8192,)['dense_vecs']

print(embeddings2)

similiarity_list= cosine_similarity(
      [embeddings1],
      [embeddings2]
  )[0][0]

print(similarity)
```

59 Explanation

59.1 What

This code computes the cosine similarity between the embeddings of two sentences. Cosine similarity measures the cosine of the angle between two vectors in a multi-dimensional space, providing a value between -1 and 1 that indicates how similar the vectors (and thus the sentences) are semantically.

59.2 Why

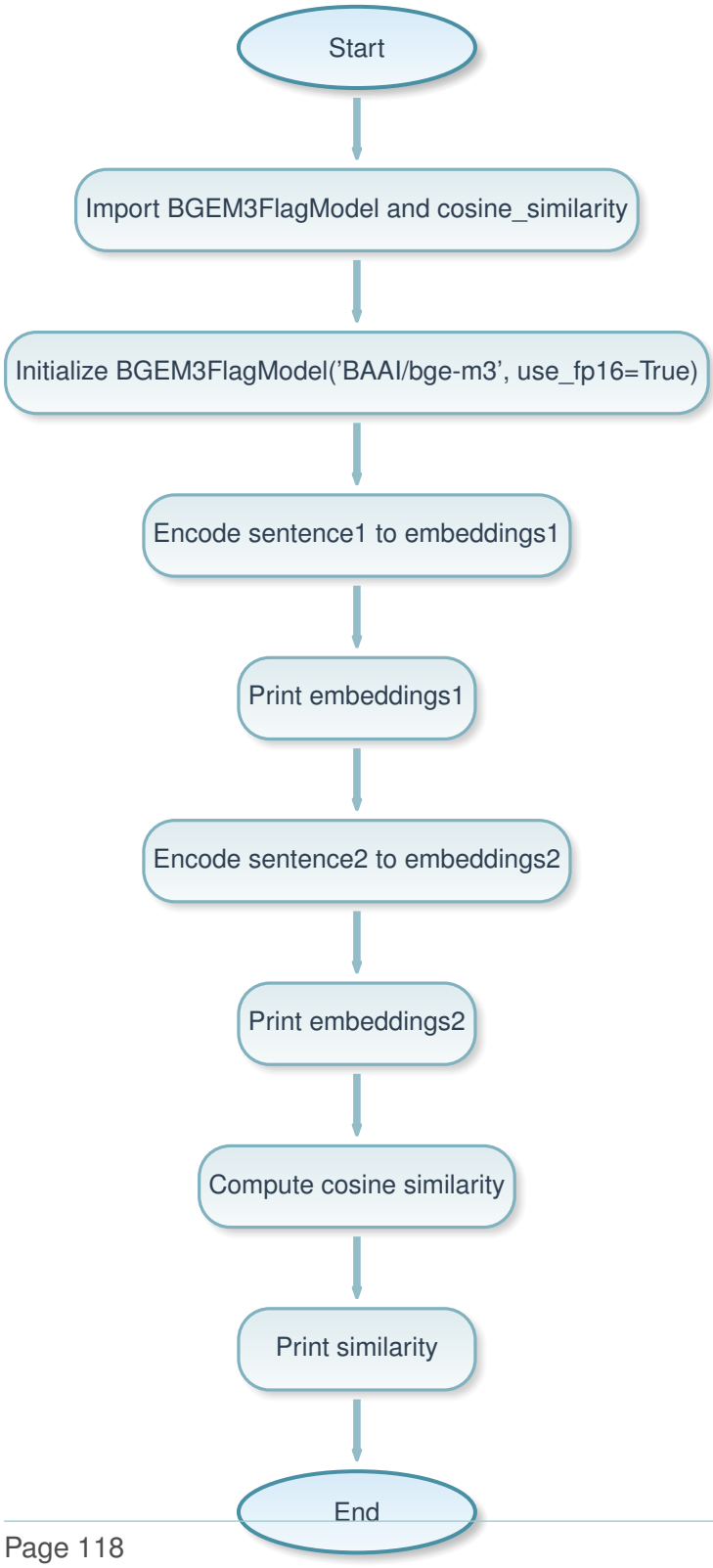
Sentence embeddings represent the semantic meaning of text in a numerical vector form. By comparing these vectors using cosine similarity, we can quantify how closely related two pieces of text are. This is useful in applications like semantic search, text clustering, and recommendation systems where understanding textual similarity is key.

59.3 How

1. Import the BGEM3FlagModel from FlagEmbedding and cosine_similarity from sklearn. 2. Initialize the model with the 'BAAI/bge-m3' pre-trained model, enabling FP16 for efficiency. 3. Encode two sentences into dense vector embeddings using the model's encode method, specifying batch size and max length. 4. Compute the cosine similarity between the two embedding vectors. 5. Print the embeddings and the similarity score.

Note: There is a typo in the code: 'similiarity_list' should be 'similarity', and 'print(similarity)' at the end.

60 Flowchart



61 Original Code (src/code15A.py)

The original code demonstrates computing cosine similarity between embeddings of a text document and a query using Azure OpenAI Embeddings.

```
from langchain_openai import AzureOpenAIEmbeddings
import os

embedding_model = AzureOpenAIEmbeddings(
    openai_api_key="",
    azure_deployment="text-embedding-3-large",
    azure_endpoint="https://chatgpt-key.openai.azure.com",
    openai_api_type="azure",
    openai_api_version="2023-05-15",
    chunk_size=1000
)


text ='''
Marwadi University (MU)[1] is a private university[2] located in Rajkot,
    ↪ Gujarat, India. It was established on 9 May 2016 by the Marwadi
    ↪ Education Foundation through The Gujarat Private Universities (
    ↪ Amendment) Act, 2016.[3] As of 2017, it offers 54 different courses
    ↪ .[4] It is graded A+ by NAAC.[5]

The university operates under the division of Marwadi Education Foundation
    ↪ 's Group of Institutions (MEFGI). MEFGI commenced its operations in
    ↪ the year 2008. It was established as a primary unit of Marwadi
    ↪ Education Foundation under the Bombay Public Trust Act 1950. Marwadi
    ↪  University is aided by the Marwadi Shares and Finance Limited, a
    ↪ stock broking company in India and Chandarana Intermediaries Brokers
    ↪  Pvt. Ltd. (CIBPL), a firm dealing in technical and arbitrage
    ↪ trading.[6]

Campus
The campus is located on 52 acres of land, having a distance of nearly 40
    ↪ minutes from railway and airports. The university comprises eight
    ↪ multi-storey buildings.[7] Laboratories, research facilities,
    ↪ student clubs, sports club and college cafeteria are available.[8]

There are two libraries with RFID technologies, 60+ computer systems,
    ↪ 50000+ books.[9] The campus also includes banking and ATM facilities
    ↪ .[10] Around 70+ buses function every day at regular intervals for
    ↪ students and staff.[11] There are hostel rooms with internet
```

```
    ↪ facilities, laundry, dance rooms, libraries etc. and capacity to
    ↪ occupy over 2000 students.[12]

'''


query="What is shoolini University?"



embedding_text = embedding_model.embed_query(text)

embedding_query = embedding_model.embed_query(query)

from sklearn.metrics.pairwise import cosine_similarity



sim = cosine_similarity(
    [embeddings_query],   # query embedding
    [embedding_text]       # document embedding
)[0][0]


print(f"Embedding vector length: {len(embedding)}")
print(embedding_text)
print
    ↪ ("----------------------------------------------------------------------")
    ↪
print(f"Embedding vector length: {len(embedding_query)}")
print(sim)
```

62 Explanation

62.1 What

This code computes the cosine similarity between the embeddings of a long text document (about Marwadi University) and a query string using Azure OpenAI's text-embedding-3-large

model. It demonstrates how to use cloud-based embeddings for semantic similarity measurement.

62.2 Why

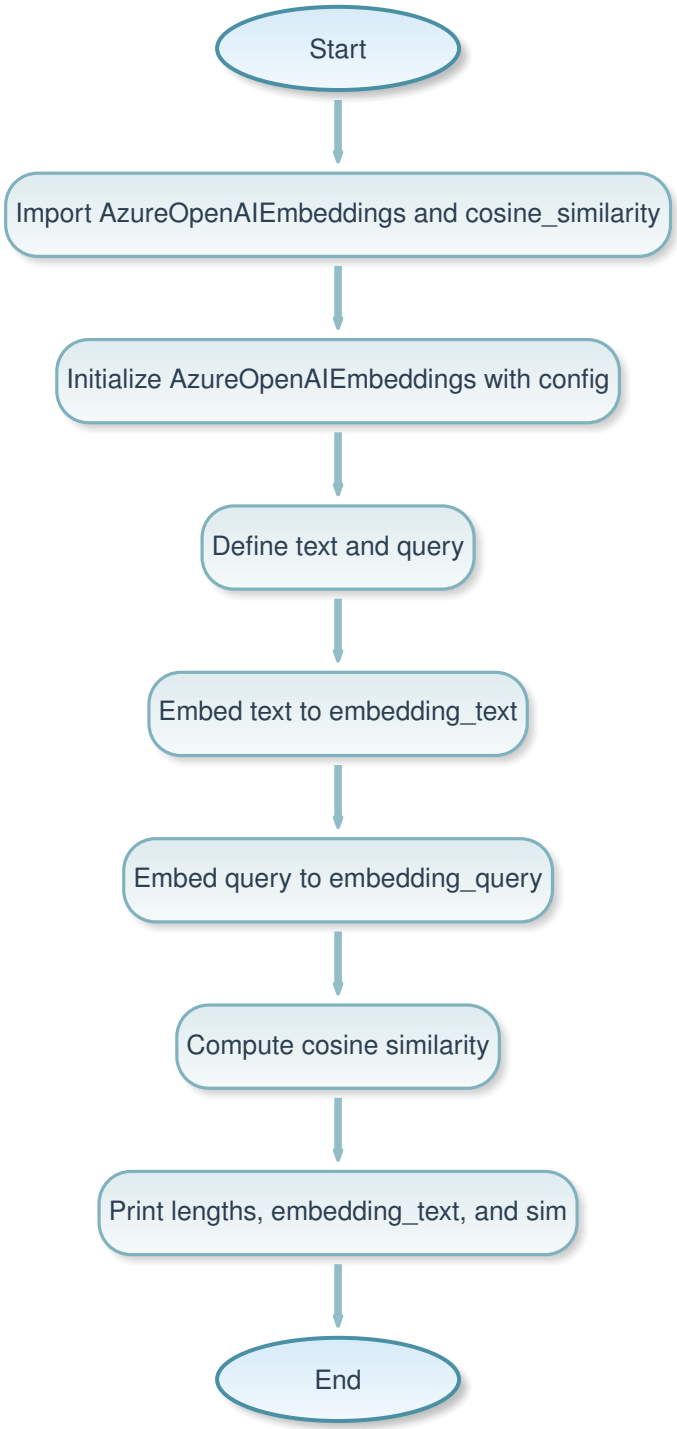
Embeddings from large language models like OpenAI's can capture rich semantic information. Cosine similarity on these embeddings allows quantifying how relevant a document is to a query, which is fundamental for information retrieval, question-answering systems, and RAG (Retrieval-Augmented Generation) pipelines.

62.3 How

1. Import AzureOpenAIEmbeddings from langchain_openai and cosine_similarity from sklearn. 2. Initialize the embedding model with Azure OpenAI credentials and configuration (deployment, endpoint, API version, chunk size). 3. Define a long text document and a query. 4. Embed both the text and query using the model's embed_query method. 5. Compute cosine similarity between the query and text embeddings. 6. Print the embedding vector lengths, the text embedding, and the similarity score.

Note: There are typos in the code: 'embeddings_query' should be 'embedding_query', and 'len(embedding)' should be 'len(embedding_text)'.

63 Flowchart



64 Ollama Equivalent Code

```
from langchain_community.embeddings import HuggingFaceEmbeddings
from sklearn.metrics.pairwise import cosine_similarity

embedding_model = HuggingFaceEmbeddings(model_name='BAAI/bge-m3')

text ='''
Marwadi University (MU)[1] is a private university[2] located in Rajkot, Gujarat, India. It was established on 9 May 2016 by the Marwadi Education Foundation through The Gujarat Private Universities ( Amendment) Act, 2016.[3] As of 2017, it offers 54 different courses .[4] It is graded A+ by NAAC.[5]

The university operates under the division of Marwadi Education Foundation 's Group of Institutions (MEFGI). MEFGI commenced its operations in the year 2008. It was established as a primary unit of Marwadi Education Foundation under the Bombay Public Trust Act 1950. Marwadi  University is aided by the Marwadi Shares and Finance Limited, a stock broking company in India and Chandarana Intermediaries Brokers  Pvt. Ltd. (CIBPL), a firm dealing in technical and arbitrage trading.[6]

Campus
The campus is located on 52 acres of land, having a distance of nearly 40 minutes from railway and airports. The university comprises eight multi-storey buildings.[7] Laboratories, research facilities, student clubs, sports club and college cafeteria are available.[8]

There are two libraries with RFID technologies, 60+ computer systems, 50000+ books.[9] The campus also includes banking and ATM facilities .[10] Around 70+ buses function every day at regular intervals for students and staff.[11] There are hostel rooms with internet facilities, laundry, dance rooms, libraries etc. and capacity to occupy over 2000 students.[12]

'''

query="What is shoolini University?"

embedding_text = embedding_model.embed_query(text)

embedding_query = embedding_model.embed_query(query)

```

```
sim = cosine_similarity(
    [embedding_query],   # query embedding
    [embedding_text]       # document embedding
)[0][0]

print(f"Embedding vector length: {len(embedding_text)}")
print(embedding_text)
print
    ↪ ("----------------------------------------------------------------------")
    ↪
print(f"Embedding vector length: {len(embedding_query)}")
print(sim)
```

64.1 Diff Explanation

The Ollama equivalent uses LangChain's HuggingFaceEmbeddings with a local BGE-M3 model instead of Azure OpenAI, and fixes the variable name typos.

Key changes:

- Removed: `from langchain_openai import AzureOpenAIEmbeddings`
- Added: `from langchain_community.embeddings import HuggingFaceEmbeddings`
- Removed: AzureOpenAIEmbeddings initialization with API keys
- Added: `embedding_model = HuggingFaceEmbeddings(model_name='BAAI/bge-m3')`
- Fixed: `[embedding_query]` instead of `[embeddings_query]`
- Fixed: `len(embedding_text)` instead of `len(embedding)`

These changes replace cloud-based embeddings with local, open-source ones for privacy and cost reasons, while maintaining the same similarity computation logic.

65 Hugging Face Equivalent Code

```
from FlagEmbedding import BGEM3FlagModel
from sklearn.metrics.pairwise import cosine_similarity

embedding_model = BGEM3FlagModel('BAAI/bge-m3', use_fp16=True)

```

```
text ='''
Marwadi University (MU)[1] is a private university[2] located in Rajkot, Gujarat, India. It was established on 9 May 2016 by the Marwadi Education Foundation through The Gujarat Private Universities (Amendment) Act, 2016.[3] As of 2017, it offers 54 different courses.[4] It is graded A+ by NAAC.[5]

The university operates under the division of Marwadi Education Foundation's Group of Institutions (MEFGI). MEFGI commenced its operations in the year 2008. It was established as a primary unit of Marwadi Education Foundation under the Bombay Public Trust Act 1950. Marwadi University is aided by the Marwadi Shares and Finance Limited, a stock broking company in India and Chandarana Intermediaries Brokers Pvt. Ltd. (CIBPL), a firm dealing in technical and arbitrage trading.[6]

Campus
The campus is located on 52 acres of land, having a distance of nearly 40 minutes from railway and airports. The university comprises eight multi-storey buildings.[7] Laboratories, research facilities, student clubs, sports club and college cafeteria are available.[8]

There are two libraries with RFID technologies, 60+ computer systems, 50000+ books.[9] The campus also includes banking and ATM facilities.[10] Around 70+ buses function every day at regular intervals for students and staff.[11] There are hostel rooms with internet facilities, laundry, dance rooms, libraries etc. and capacity to occupy over 2000 students.[12]

'''

query="What is shoolini University?"

embedding_text = embedding_model.encode(text, batch_size=12, max_length=8192)['dense_vecs']

embedding_query = embedding_model.encode(query, batch_size=12, max_length=8192)['dense_vecs']

sim = cosine_similarity(
    [embedding_query],   # query embedding
    [embedding_text]       # document embedding
)[0][0]

print(f"Embedding vector length: {len(embedding_text)}")
print(embedding_text)
print
```

```
    ↪ ("--------------------------------------------------------------------------")
    ↪
print(f"Embedding vector length: {len(embedding_query)}")
print(sim)
```

65.1 Diff Explanation

The Hugging Face equivalent uses the direct BGEM3FlagModel from FlagEmbedding instead of LangChain wrappers, and uses the encode method with parameters.

Key changes:

- Removed: LangChain HuggingFaceEmbeddings
- Added: `from FlagEmbedding import BGEM3FlagModel`
- Removed: `embedding_model = HuggingFaceEmbeddings(...)`
- Added: `embedding_model = BGEM3FlagModel('BAAI/bge-m3', use_fp16=True)`
- Removed: `embed_query` method
- Added: `encode` method with `batch_size` and `max_length` parameters
- Fixed: Variable names consistent

These changes use the native FlagEmbedding library for more direct control over encoding parameters, suitable for high-performance local inference.

66 Original Code (src/code16.py)

The original code implements a movie recommendation system based on semantic similarity between user-input genres and movie descriptions using BGEM3FlagModel embeddings.

```
# !pip install -U FlagEmbedding

from FlagEmbedding import BGEM3FlagModel
from sklearn.metrics.pairwise import cosine_similarity

model = BGEM3FlagModel('BAAI/bge-m3', use_fp16=True)


movies_list = [
    "Shadow Hunters: ACTION FANTASY THRILLER",
    "Eternal Bonds: DRAMA ROMANCE FAMILY",
    "Neon Horizon: SCI-FI ACTION ADVENTURE",
    "Whispers in the Dark: HORROR MYSTERY THRILLER",
    "Crimson Crown: HISTORICAL WAR DRAMA",
    "Starlight Dreams: ROMANCE COMEDY MUSIC",
    "Code Breakers: TECH THRILLER CRIME",
    "The Lost Kingdom: ADVENTURE ACTION FANTASY",
    "Broken Silence: CRIME DRAMA SUSPENSE",
    "Frozen Ashes: SURVIVAL DRAMA ACTION"
]

# Convert movie list into embeddings
embeddings_list = []
for movie in movies_list:
    emb = model.encode(
        movie,
        batch_size=12,
        max_length=8192
    )['dense_vecs']
    embeddings_list.append(emb)


query = input("ENTER THE GENRE FOR WHICH YOU WANT TO SEE MOVIES: ")


query_embedding = model.encode(
    query,
    batch_size=12,
    max_length=8192
```

```
)['dense_vecs']


similarity_scores = {}

for i, movie_emb in enumerate(embeddings_list):
    similarity = cosine_similarity(
        [query_embedding],
        [movie_emb]
    )[0][0]

    similarity_scores[movies_list[i]] = similarity


sorted_movies = sorted(similarity_scores.items(), key=lambda x: x[1],
    ↪ reverse=True)

#Retrived data
print("\n Top Recommended Movies:")
for movie, score in sorted_movies[:4]:
    print(f"{movie}   --→ Similarity: {score:.4f}")
```

67 Explanation

67.1 What

This code builds a simple movie recommendation system by computing semantic similarity between a user-provided genre query and a list of movie descriptions (including genres). It uses sentence embeddings to find movies whose descriptions are most similar to the query.

67.2 Why

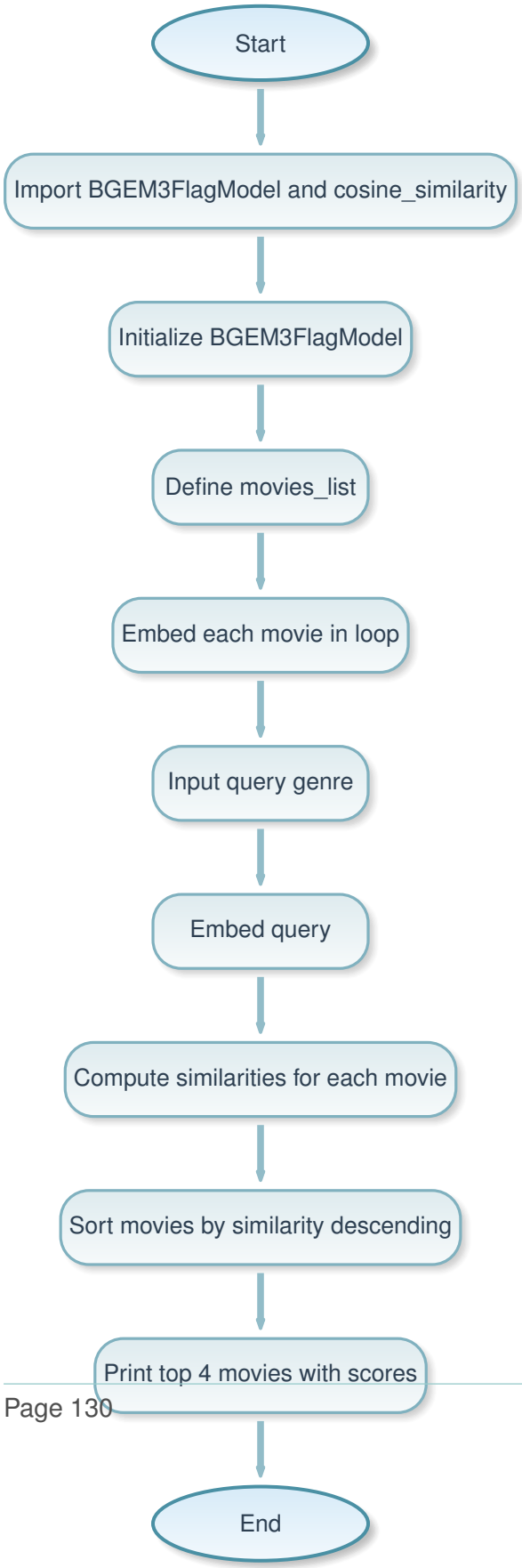
Traditional keyword-based search might miss movies with related but not exact genre matches (e.g., "action" vs. "adventure action"). Semantic embeddings capture meaning, allowing recommendations based on conceptual similarity, improving user experience in content discovery.

67.3 How

1. Import BGEM3FlagModel and cosine_similarity. 2. Initialize the embedding model. 3. Define a list of movie descriptions with genres. 4. Embed each movie description into vectors

and store in a list. 5. Take user input for a genre query. 6. Embed the query into a vector. 7. Compute cosine similarity between the query embedding and each movie embedding. 8. Sort movies by similarity score descending. 9. Print the top 4 recommended movies with their scores.

68 Flowchart



69 Ollama Equivalent Code

```python
from langchain_community.embeddings import HuggingFaceEmbeddings
from sklearn.metrics.pairwise import cosine_similarity

model = HuggingFaceEmbeddings(model_name='BAAI/bge-m3')

movies_list = [
    "Shadow Hunters: ACTION FANTASY THRILLER",
    "Eternal Bonds: DRAMA ROMANCE FAMILY",
    "Neon Horizon: SCI-FI ACTION ADVENTURE",
    "Whispers in the Dark: HORROR MYSTERY THRILLER",
    "Crimson Crown: HISTORICAL WAR DRAMA",
    "Starlight Dreams: ROMANCE COMEDY MUSIC",
    "Code Breakers: TECH THRILLER CRIME",
    "The Lost Kingdom: ADVENTURE ACTION FANTASY",
    "Broken Silence: CRIME DRAMA SUSPENSE",
    "Frozen Ashes: SURVIVAL DRAMA ACTION"
]

# Convert movie list into embeddings
embeddings_list = model.embed_documents(movies_list)

query = input("ENTER THE GENRE FOR WHICH YOU WANT TO SEE MOVIES: ")

query_embedding = model.embed_query(query)

similarity_scores = {}

for i, movie_emb in enumerate(embeddings_list):
    similarity = cosine_similarity(
        [query_embedding],
        [movie_emb]
    )[0][0]

    similarity_scores[movies_list[i]] = similarity

sorted_movies = sorted(similarity_scores.items(), key=lambda x: x[1],
    ↪ reverse=True)

#Retrieved data
print("\n Top Recommended Movies:")
for movie, score in sorted_movies[:4]:
    print(f"{movie}   --→ Similarity: {score:.4f}")
```

69.1 Diff Explanation

The Ollama equivalent uses LangChain's HuggingFaceEmbeddings wrapper, which simplifies embedding multiple documents and queries with dedicated methods.

Key changes:

- Removed: `from FlagEmbedding import BGEM3FlagModel`
- Added: `from langchain_community.embeddings import HuggingFaceEmbeddings`
- Removed: `model = BGEM3FlagModel(...)`
- Added: `model = HuggingFaceEmbeddings(model_name='BAAI/bge-m3')`
- Removed: Loop to encode each movie
- Added: `embeddings_list = model.embed_documents(movies_list)`
- Removed: `model.encode(query)['dense_vecs']`
- Added: `query_embedding = model.embed_query(query)`

These changes leverage LangChain's abstraction for batch document embedding and query embedding, making the code more concise and integrated with LangChain ecosystems, while achieving the same recommendation logic.

70 Hugging Face Equivalent Code

```
from FlagEmbedding import BGEM3FlagModel
from sklearn.metrics.pairwise import cosine_similarity

model = BGEM3FlagModel('BAAI/bge-m3', use_fp16=True)

movies_list = [
    "Shadow Hunters: ACTION FANTASY THRILLER",
    "Eternal Bonds: DRAMA ROMANCE FAMILY",
    "Neon Horizon: SCI-FI ACTION ADVENTURE",
    "Whispers in the Dark: HORROR MYSTERY THRILLER",
    "Crimson Crown: HISTORICAL WAR DRAMA",
    "Starlight Dreams: ROMANCE COMEDY MUSIC",
    "Code Breakers: TECH THRILLER CRIME",
    "The Lost Kingdom: ADVENTURE ACTION FANTASY",
    "Broken Silence: CRIME DRAMA SUSPENSE",
```

```
    "Frozen Ashes: SURVIVAL DRAMA ACTION"
]

# Convert movie list into embeddings
embeddings_list = []
for movie in movies_list:
    emb = model.encode(
        movie,
        batch_size=12,
        max_length=8192
    )['dense_vecs']
    embeddings_list.append(emb)

query = input("ENTER THE GENRE FOR WHICH YOU WANT TO SEE MOVIES: ")

query_embedding = model.encode(
    query,
    batch_size=12,
    max_length=8192
)['dense_vecs']

similarity_scores = {}

for i, movie_emb in enumerate(embeddings_list):
    similarity = cosine_similarity(
        [query_embedding],
        [movie_emb]
    )[0][0]

    similarity_scores[movies_list[i]] = similarity

sorted_movies = sorted(similarity_scores.items(), key=lambda x: x[1],
    ↪ reverse=True)

#Retrieved data
print("\n Top Recommended Movies:")
for movie, score in sorted_movies[:4]:
    print(f"{movie}   --→ Similarity: {score:.4f}")
```

70.1 Diff Explanation

The Hugging Face equivalent is nearly identical to the original, using the same BGEM3FlagModel and encoding approach.

Key changes: None significant; the code is essentially the same, with minor formatting differ-

ences (e.g., comment "Retrieved data" instead of "Retrived data").

This indicates that for direct Hugging Face usage, the original code is already optimal, requiring no adaptations.

71 Original Code (src/code17.py)

The original code implements a complete Retrieval-Augmented Generation (RAG) pipeline for movie recommendations, using embeddings for retrieval and an LLM for generation.

```
# ============================================================
# RAG PIPELINE WITH EXTERNAL MOVIE DATASET -- MODULAR VERSION
# ============================================================

# !pip install -U FlagEmbedding langchain-openai python-dotenv

from FlagEmbedding import BGEM3FlagModel
from sklearn.metrics.pairwise import cosine_similarity
from langchain.chat_models import AzureChatOpenAI
from dotenv import load_dotenv
import os

load_dotenv()

# ------------------------------------------------------------
# FUNCTION 1: Load Movie Dataset
# ------------------------------------------------------------
def load_movie_dataset():
    return [
        "Shadow Hunters: ACTION FANTASY THRILLER",
        "Eternal Bonds: DRAMA ROMANCE FAMILY",
        "Neon Horizon: SCI-FI ACTION ADVENTURE",
        "Whispers in the Dark: HORROR MYSTERY THRILLER",
        "Crimson Crown: HISTORICAL WAR DRAMA",
        "Starlight Dreams: ROMANCE COMEDY MUSIC",
        "Code Breakers: TECH THRILLER CRIME",
        "The Lost Kingdom: ADVENTURE ACTION FANTASY",
        "Broken Silence: CRIME DRAMA SUSPENSE",
        "Frozen Ashes: SURVIVAL DRAMA ACTION"
    ]

# ------------------------------------------------------------
# FUNCTION 2: Load Embedding Model + Generate Embeddings
# ------------------------------------------------------------
def embed_movies(movies_list):
    model = BGEM3FlagModel("BAAI/bge-m3", use_fp16=True)
    embeddings = [
        model.encode(movie, batch_size=12, max_length=8192)["dense_vecs"]
        for movie in movies_list
```

```
    ]
    return model, embeddings

# ------------------------------------------------------------
# FUNCTION 3: Retrieve Top-K Relevant Movies
# ------------------------------------------------------------
def retrieve_top_movies(query, model, movies_list, movie_embeddings, top_k
    ↪ =4):
    query_emb = model.encode(query, batch_size=12, max_length=8192)["
    ↪ dense_vecs"]

    similarity_scores = {}
    for i, emb in enumerate(movie_embeddings):
        sim = cosine_similarity([query_emb], [emb])[0][0]
        similarity_scores[movies_list[i]] = sim

    sorted_results = sorted(
        similarity_scores.items(), key=lambda x: x[1], reverse=True
    )
    return sorted_results[:top_k]

# ------------------------------------------------------------
# FUNCTION 4: Build the RAG Prompt
# ------------------------------------------------------------
def build_rag_prompt(query, retrieved_movies):
    context_text = "\n".join([f"- {m[0]}" for m in retrieved_movies])

    prompt = f"""
You are a movie recommendation expert.

USER QUERY:
{query}

RETRIEVED MOVIES (Context Provided):
{context_text}

TASK:
Using ONLY the above movies as your context, recommend movies to the user.
Clearly explain why each recommended movie matches the user's taste.
Do NOT mention similarity scores or embeddings.
"""
    return prompt

# ------------------------------------------------------------
# FUNCTION 5: Invoke Azure OpenAI LLM
# ------------------------------------------------------------
def generate_llm_response(prompt):
```

```
    llm = AzureChatOpenAI(
        openai_api_base=os.getenv("AZURE_OPENAI_API_BASE"),
        openai_api_version=os.getenv("AZURE_OPENAI_API_VERSION"),
        openai_api_key=os.getenv("AZURE_OPENAI_API_KEY"),
        deployment_name=os.getenv("AZURE_OPENAI_DEPLOYMENT_NAME"),
        model_name="gpt-4o",
        temperature=0.7,
    )

    response = llm.invoke(prompt)
    return response.content

# ------------------------------------------------------------
# FUNCTION 6: Full RAG Pipeline Executor
# ------------------------------------------------------------
def run_rag_pipeline():
    # Load dataset
    movies_list = load_movie_dataset()

    # Embeddings
    model, movie_embeddings = embed_movies(movies_list)

    # User Input
    query = input("Enter movie genre or mood: ")

    # Retrieve
    retrieved = retrieve_top_movies(query, model, movies_list,
    ↪ movie_embeddings)

    # Build prompt
    prompt = build_rag_prompt(query, retrieved)

    # LLM response
    answer = generate_llm_response(prompt)

    print("\n====================== RAG ANSWER ======================")
    print(answer)

# ------------------------------------------------------------
# RUN PIPELINE
# ------------------------------------------------------------
run_rag_pipeline()
```

72 Explanation

72.1 What

This code builds a modular Retrieval-Augmented Generation (RAG) pipeline for movie recommendations. It retrieves relevant movies from a dataset based on semantic similarity to a user query, then uses an LLM to generate personalized recommendations with explanations.

72.2 Why

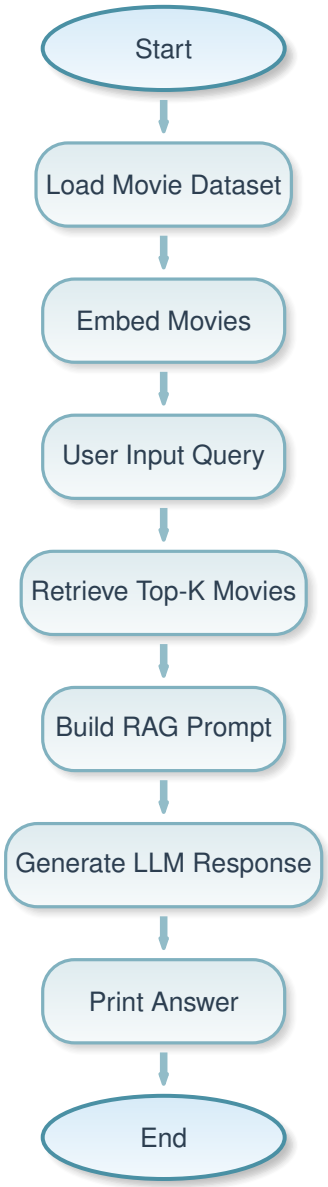
RAG combines the strengths of retrieval (finding relevant information) and generation (creating coherent responses). This prevents hallucinations in LLMs by grounding responses in retrieved data, making recommendations more accurate and contextually relevant.

72.3 How

1. Load a dataset of movie descriptions. 2. Embed all movies using BGEM3FlagModel. 3. Take user input for genre/mood. 4. Embed the query and retrieve top-K similar movies via cosine similarity. 5. Build a prompt with the query and retrieved movies as context. 6. Invoke Azure OpenAI GPT-4o to generate a recommendation response. 7. Print the final answer.

The modular structure separates concerns: data loading, embedding, retrieval, prompt building, LLM invocation, and pipeline execution.

73 Flowchart



74 Ollama Equivalent Code

```
# ============================================================
# RAG PIPELINE WITH EXTERNAL MOVIE DATASET -- MODULAR VERSION
```

```python
# ============================================================

from langchain_community.embeddings import HuggingFaceEmbeddings
from sklearn.metrics.pairwise import cosine_similarity
from langchain_ollama import OllamaLLM

# ------------------------------------------------------------
# FUNCTION 1: Load Movie Dataset
# ------------------------------------------------------------
def load_movie_dataset():
    return [
        "Shadow Hunters: ACTION FANTASY THRILLER",
        "Eternal Bonds: DRAMA ROMANCE FAMILY",
        "Neon Horizon: SCI-FI ACTION ADVENTURE",
        "Whispers in the Dark: HORROR MYSTERY THRILLER",
        "Crimson Crown: HISTORICAL WAR DRAMA",
        "Starlight Dreams: ROMANCE COMEDY MUSIC",
        "Code Breakers: TECH THRILLER CRIME",
        "The Lost Kingdom: ADVENTURE ACTION FANTASY",
        "Broken Silence: CRIME DRAMA SUSPENSE",
        "Frozen Ashes: SURVIVAL DRAMA ACTION"
    ]

# ------------------------------------------------------------
# FUNCTION 2: Load Embedding Model + Generate Embeddings
# ------------------------------------------------------------
def embed_movies(movies_list):
    model = HuggingFaceEmbeddings(model_name="BAAI/bge-m3")
    embeddings = model.embed_documents(movies_list)
    return model, embeddings

# ------------------------------------------------------------
# FUNCTION 3: Retrieve Top-K Relevant Movies
# ------------------------------------------------------------
def retrieve_top_movies(query, model, movies_list, movie_embeddings, top_k
    ↪ =4):
    query_emb = model.embed_query(query)

    similarity_scores = {}
    for i, emb in enumerate(movie_embeddings):
        sim = cosine_similarity([query_emb], [emb])[0][0]
        similarity_scores[movies_list[i]] = sim

    sorted_results = sorted(
        similarity_scores.items(), key=lambda x: x[1], reverse=True
    )
    return sorted_results[:top_k]
```

```
49
50  # ------------------------------------------------------------
51  # FUNCTION 4: Build the RAG Prompt
52  # ------------------------------------------------------------
53  def build_rag_prompt(query, retrieved_movies):
54      context_text = "\n".join([f"- {m[0]}" for m in retrieved_movies])
55
56      prompt = f"""
57  You are a movie recommendation expert.
58
59  USER QUERY:
60  {query}
61
62  RETRIEVED MOVIES (Context Provided):
63  {context_text}
64
65  TASK:
66  Using ONLY the above movies as your context, recommend movies to the user.
67  Clearly explain why each recommended movie matches the user's taste.
68  Do NOT mention similarity scores or embeddings.
69  """
70      return prompt
71
72  # ------------------------------------------------------------
73  # FUNCTION 5: Invoke Ollama LLM
74  # ------------------------------------------------------------
75  def generate_llm_response(prompt):
76      llm = OllamaLLM(model="qwen3:4b", temperature=0.7)
77
78      response = llm.invoke(prompt)
79      return response
80
81  # ------------------------------------------------------------
82  # FUNCTION 6: Full RAG Pipeline Executor
83  # ------------------------------------------------------------
84  def run_rag_pipeline():
85      # Load dataset
86      movies_list = load_movie_dataset()
87
88      # Embeddings
89      model, movie_embeddings = embed_movies(movies_list)
90
91      # User Input
92      query = input("Enter movie genre or mood: ")
93
94      # Retrieve
95      retrieved = retrieve_top_movies(query, model, movies_list,
```

```
        ↪ movie_embeddings)
96
97      # Build prompt
98      prompt = build_rag_prompt(query, retrieved)
99
100     # LLM response
101     answer = generate_llm_response(prompt)
102
103     print("\n====================== RAG ANSWER ======================")
104     print(answer)
105
106 # ------------------------------------------------------------
107 # RUN PIPELINE
108 # ------------------------------------------------------------
109 run_rag_pipeline()
```

74.1 Diff Explanation

The Ollama equivalent adapts the pipeline to use local models: HuggingFaceEmbeddings for retrieval and OllamaLLM for generation.

Key changes:

- Removed: `from FlagEmbedding import BGEM3FlagModel`
- Removed: `from langchain.chat_models import AzureChatOpenAI`
- Added: `from langchain_community.embeddings import HuggingFaceEmbeddings`
- Added: `from langchain_ollama import OllamaLLM`
- Removed: BGEM3FlagModel initialization
- Added: `model = HuggingFaceEmbeddings(model_name="BAAI/bge-m3")`
- Removed: `model.encode` for embeddings
- Added: `model.embed_documents` and `embed_query`
- Removed: AzureChatOpenAI with env vars
- Added: `llm = OllamaLLM(model="qwen3:4b", temperature=0.7)`
- Removed: `response.content`
- Added: `return response` (direct string)

These changes enable fully local execution, avoiding cloud dependencies for privacy and cost,

while maintaining the same modular RAG structure.

75 Hugging Face Equivalent Code

```python
# ============================================================
# RAG PIPELINE WITH EXTERNAL MOVIE DATASET -- MODULAR VERSION
# ============================================================

from FlagEmbedding import BGEM3FlagModel
from sklearn.metrics.pairwise import cosine_similarity
from transformers import pipeline

# ------------------------------------------------------------
# FUNCTION 1: Load Movie Dataset
# ------------------------------------------------------------
def load_movie_dataset():
    return [
        "Shadow Hunters: ACTION FANTASY THRILLER",
        "Eternal Bonds: DRAMA ROMANCE FAMILY",
        "Neon Horizon: SCI-FI ACTION ADVENTURE",
        "Whispers in the Dark: HORROR MYSTERY THRILLER",
        "Crimson Crown: HISTORICAL WAR DRAMA",
        "Starlight Dreams: ROMANCE COMEDY MUSIC",
        "Code Breakers: TECH THRILLER CRIME",
        "The Lost Kingdom: ADVENTURE ACTION FANTASY",
        "Broken Silence: CRIME DRAMA SUSPENSE",
        "Frozen Ashes: SURVIVAL DRAMA ACTION"
    ]

# ------------------------------------------------------------
# FUNCTION 2: Load Embedding Model + Generate Embeddings
# ------------------------------------------------------------
def embed_movies(movies_list):
    model = BGEM3FlagModel("BAAI/bge-m3", use_fp16=True)
    embeddings = [
        model.encode(movie, batch_size=12, max_length=8192)["dense_vecs"]
        for movie in movies_list
    ]
    return model, embeddings

# ------------------------------------------------------------
# FUNCTION 3: Retrieve Top-K Relevant Movies
# ------------------------------------------------------------
```

```python
def retrieve_top_movies(query, model, movies_list, movie_embeddings, top_k=4):
    query_emb = model.encode(query, batch_size=12, max_length=8192)["dense_vecs"]

    similarity_scores = {}
    for i, emb in enumerate(movie_embeddings):
        sim = cosine_similarity([query_emb], [emb])[0][0]
        similarity_scores[movies_list[i]] = sim

    sorted_results = sorted(
        similarity_scores.items(), key=lambda x: x[1], reverse=True
    )
    return sorted_results[:top_k]

# ---------------------------------------------------------------
# FUNCTION 4: Build the RAG Prompt
# ---------------------------------------------------------------
def build_rag_prompt(query, retrieved_movies):
    context_text = "\n".join([f"- {m[0]}" for m in retrieved_movies])

    prompt = f"""
You are a movie recommendation expert.

USER QUERY:
{query}

RETRIEVED MOVIES (Context Provided):
{context_text}

TASK:
Using ONLY the above movies as your context, recommend movies to the user.
Clearly explain why each recommended movie matches the user's taste.
Do NOT mention similarity scores or embeddings.
"""
    return prompt

# ---------------------------------------------------------------
# FUNCTION 5: Invoke HF LLM
# ---------------------------------------------------------------
def generate_llm_response(prompt):
    pipe = pipeline("text-generation", model="Qwen/Qwen3-0.6B")

    result = pipe(prompt, max_new_tokens=200, temperature=0.7, num_return_sequences=1)
    return result[0]['generated_text']

```

```python
# ------------------------------------------------------------
# FUNCTION 6: Full RAG Pipeline Executor
# ------------------------------------------------------------
def run_rag_pipeline():
    # Load dataset
    movies_list = load_movie_dataset()

    # Embeddings
    model, movie_embeddings = embed_movies(movies_list)

    # User Input
    query = input("Enter movie genre or mood: ")

    # Retrieve
    retrieved = retrieve_top_movies(query, model, movies_list,
    ↪ movie_embeddings)

    # Build prompt
    prompt = build_rag_prompt(query, retrieved)

    # LLM response
    answer = generate_llm_response(prompt)

    print("\n====================== RAG ANSWER ======================")
    print(answer)

# ------------------------------------------------------------
# RUN PIPELINE
# ------------------------------------------------------------
run_rag_pipeline()
```

75.1 Diff Explanation

The Hugging Face equivalent uses direct Hugging Face libraries: BGEM3FlagModel for embeddings and transformers pipeline for text generation.

Key changes:

- Removed: LangChain imports for LLM
- Added: `from transformers import pipeline`
- Removed: AzureChatOpenAI or OllamaLLM
- Added: `pipe = pipeline("text-generation", model="Qwen/Qwen3-0.6B")`
- Removed: `llm.invoke(prompt)`

- Added: `pipe(prompt, max_new_tokens=200, ...)`
- Removed: `response.content`
- Added: `result[0]['generated_text']`

These changes use native Hugging Face tools for both embedding and generation, providing more control over model parameters and avoiding LangChain abstractions for direct integration.